[Nam Le] IPC: Pipes, FIFOs, Message Queues, Shared Memory, Signals

1. Pipes, FIFO, Message Queues

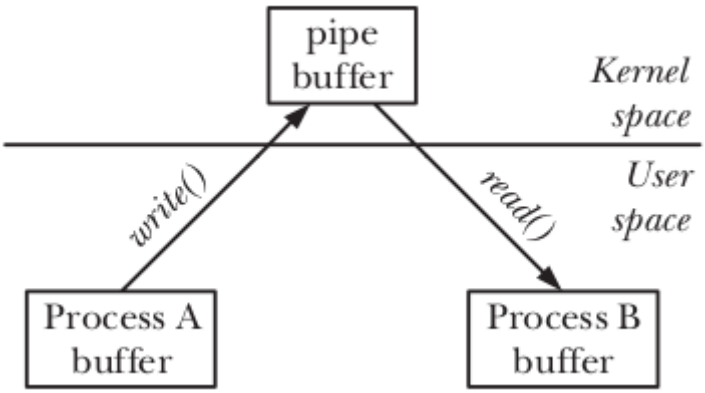
1.1 What is IPC?

IPC (InterProcess communication) is general term, with 3 function:

- Communication: exchanging data between processes
- Synchronization: synchronizing the actions of processes or threads
- Signals: can be used as a synchronization technique in certain circumstances

1. Communication facilities:

Data-transfer: In order to communicate, one process writes data to the IPC facility, and another process reads the data. One transfer from user memory to kernel memory during writing, and another transfer from kernel memory to user memory during reading. Read is **constructive**



Share-memory: exchange information by placing it in a region of memory that is shared between the processes. A process can make data available to other processes by placing it in the shared memory region

With **Data-Transfer**, we have subcategories:

- Byte stream: The data exchanged via pipes, FIFOs, and datagram sockets **is an undelimited byte stream**. Each read operation may read an arbitrary number of bytes from the IPC facility, regardless of the size of blocks written by the writer.
- The data exchanged via System V message queues, POSIX message queues, and datagram sockets takes the form of delimited messages
- Pseudoterminals (specical situations): emulator Terminal Device

With **Shared memory**:

- shared memory provides fast communication (because dont require system call or exchange by kernel)
- Data placed in shared memory is visible to all of the processes that share that memory

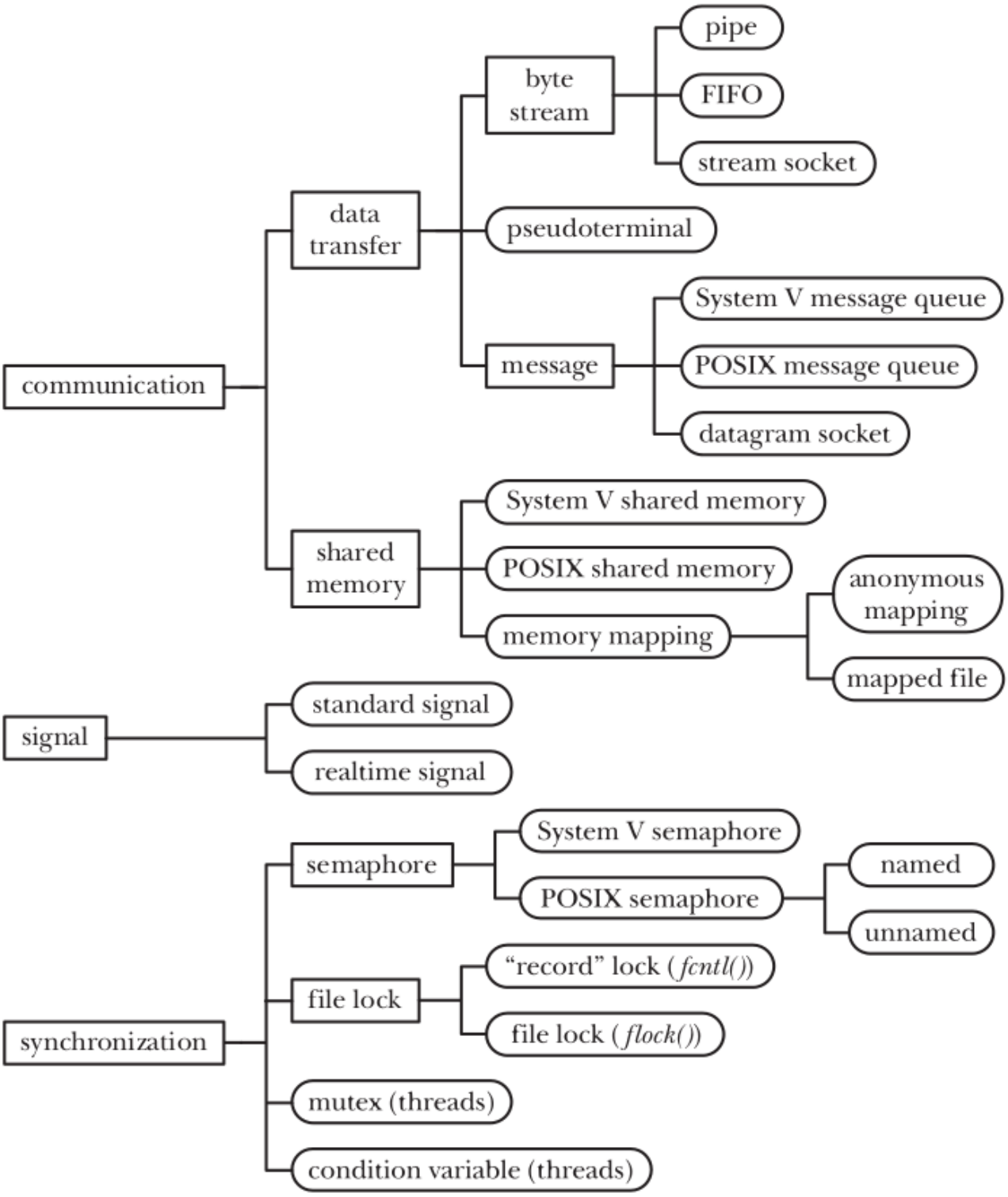


Figure 43-1: A taxonomy of UNIX IPC facilities

1.2 Pipe

Ex: `$ ls | wc -l`
In order to execute the above command, the shell creates two processes, executing `ls` and `wc` (done using `fork()` and `exec()`)
child will do `ls` and has its standard output (stdout)

`|` is a pipe which connect stdout of `ls` with stdin of `wc -l`
another child is created will do `wc -l` to count the line from output of `ls`

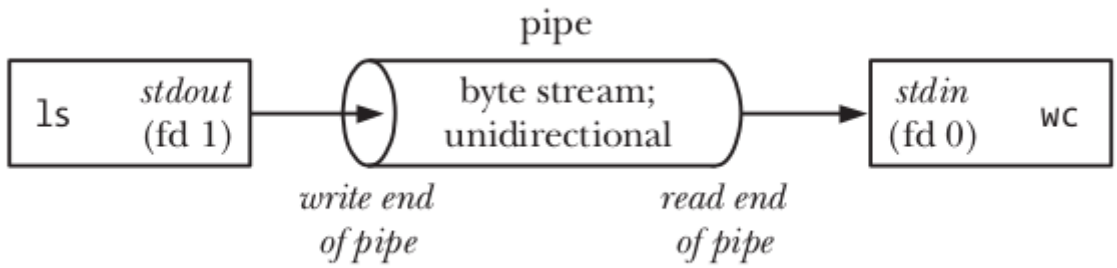


Figure 44-1: Using a pipe to connect two processes

The example code equal `ls | wc -l`:

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {     /*Create pipe*/
        perror("Failure on create pipe");
        exit(1);
    }
    pid1 = fork();
    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO); /* Connect stdout of this process to the write of pipe
*/

        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", NULL); /* Overwrite by execlp */
        perror("exec fail");
        exit(1);
    }
    pid2 = fork();
    if (pid2 == 0) {
        dup2(pipefd[0], STDIN_FILENO); /* Connect the read end of pipe to stdin of this
```

```
process*/

        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL); /* Overwrite by execlp */
        perror("exec fail");
        exit(1);
    }
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

Overview feature of **PIPE**:
A pipe is a byte stream:No limit size of the process reading, data pass through pipe sequentially—bytes

Reading from a pipe: Read from a pipe that is currently block until at least one byte has been written to the pipe. If the wrtie end of file of a pipe is closed, then **reading** process from a pipe will see **end-of-file** (read() return 0)

Pipes are unidirectional: One end of pipe using for writing, and the other used for reading

Writes of up to PIPE_BUF bytes are guaranteed to be atomic:
Example about PIPE_BUF:

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {     /*Create pipe*/
```

```
        perror("Failure on create pipe");
        exit(1);
    }
    pid1 = fork();
    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO); /* Connect stdout of this process to the write of pipe
*/
        /*Copied success, close head and tail of pipe*/
        close(pipefd[1]);

        char *msg = "Dong1\nDong2\nDong3\n";
        ssize_t n = write(STDOUT_FILENO, msg, strlen(msg));
        fprintf(stderr, "[Write] Wrote %ld bytes into pipe\n",n);

        exit(0);
    }
    pid2 = fork();
    if (pid2 == 0) {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO); /* Connect the read end of pipe to stdin of this
process*/
        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);

        char buffer[8];   //PIPE_BUF
        ssize_t bytes_read;
        while ((bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0) {
            buffer[bytes_read] = '\0';
            printf("[Read] Received %ld bytes: \"%s\"\n", bytes_read, buffer);
        }
        exit(0);
    }
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

Result:

```
[Write] Wrote 18 bytes into pipe
[Read] Received 7 bytes: "Dong1
D"
[Read] Received 7 bytes: "ong2
Do"
[Read] Received 4 bytes: "ng3
"
```

--> It means that if data larger than PIPE_BUF bytes (**writer**), kernel tranfers data in multiple smaller pieces

But, With the theory:

> However, if there are multiple writer processes, then writes of large blocks **may be** broken into segments of arbitrary size (which may be smaller than PIPE_BUF bytes) and interleaved with writes by other processes

May be theory is wrong . (Addition: I'm wrong. Buffer size is only **8** < **PIPE_BUF (4096 bytes)** --> still remain **atomic**)
Example (multiple writers):

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
int main() {
    int pipefd[2];
    pid_t pid1, pid2, pid3;

    if (pipe(pipefd) == -1) {     /*Create pipe*/
        perror("Failure on create pipe");
        exit(1);
    }
    pid1 = fork();
    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO); /* Connect stdout of this process to the write of pipe
*/
```

```
        /*Copied success, close head and tail of pipe*/
        close(pipefd[1]);

        char *msg = "Dong1\nDong2\nDong3\n";
        ssize_t n = write(STDOUT_FILENO, msg, strlen(msg));
        fprintf(stderr, "[Write1] Wrote %ld bytes into pipe\n",n);

        exit(0);
    }
    pid3 = fork();
    if (pid3 == 0) {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        char *msg = "Dong4\nDong5\nDong6\n";
        ssize_t n = write(STDOUT_FILENO, msg, strlen(msg));
        fprintf(stderr, "[Write2] Wrote %ld bytes into pipe\n",n);

        exit(0);

    }
    pid2 = fork();
    if (pid2 == 0) {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO); /* Connect the read end of pipe to stdin of this
process*/
        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);

        char buffer[8];   //PIPE_BUF
        ssize_t bytes_read;
        while ((bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0) {
            buffer[bytes_read] = '\0';
            printf("[Read] Received %ld bytes: \"%s\"\n", bytes_read, buffer);
        }
        exit(0);
    }
    close(pipefd[0]);
    close(pipefd[1]);
```

```
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

Result:

```
[Write1] Wrote 18 bytes into pipe
[Write2] Wrote 18 bytes into pipe
[Read] Received 7 bytes: "Dong1
D"
[Read] Received 7 bytes: "ong2
Do"
[Read] Received 7 bytes: "ng3
Don"
[Read] Received 7 bytes: "g4
Dong"
[Read] Received 7 bytes: "5
Dong6"
[Read] Received 1 bytes: "
"
```

still **transfers as much data as possible**, not broken **into segments of arbitrary size**

Pipes have a limited capacity:

```
int fd[2];
    pipe(fd);
    char c = 'c';
    int count = 0;
    while (1) {
        write(fd[1], &c, 1);
        printf("%d byte\n", ++count);
    }
```

Result: max of pipe size is 65536 bytes (16KB)

What happen if I write the data bigger than MAX Capacity Pipe Size?

Follow the theory, it must write 64kb (65535 bytes) for each time. But see the example:

```
#define BLOCK_SIZE (PIPE_BUF * 30)  /* 122880 bytes*/

void writer(int wfd) {
    char *buf = malloc(BLOCK_SIZE);
    memset(buf, 'A', BLOCK_SIZE);
    buf[BLOCK_SIZE - 1] = '\n';

    ssize_t total = 0;
    while (total < BLOCK_SIZE) {
        ssize_t n = write(wfd, buf + total, BLOCK_SIZE - total);
        if (n <= 0) { perror("write"); exit(1); }
        total += n;
        fprintf(stderr, "[Writer] wrote %ld bytes (total %ld)\n", n, total);
    }
    free(buf);
}

int main() {
    int pipefd[2];
    if (pipe(pipefd) == -1) { perror("pipe"); exit(1); }

    pid_t w1 = fork();
    if (w1 == 0) {
        close(pipefd[0]);
        writer(pipefd[1]);
        close(pipefd[1]);
        exit(0);
    }

    close(pipefd[1]);
    char buf[4096];
    ssize_t n;
    long total = 0;
```

```
    while ((n = read(pipefd[0], buf, sizeof(buf))) > 0) {
        total += n;
        fprintf(stderr, "[Reader] read %ld bytes (total %ld), \n",n, total);
    }
    close(pipefd[0]);
    waitpid(w1, NULL, 0);
    return 0;
}
```

Result:

```
[Writer A] wrote 122880 bytes (total 122880)
[Reader] read 4096 bytes (total 4096),
[Reader] read 4096 bytes (total 8192),
[Reader] read 4096 bytes (total 12288),
[Reader] read 4096 bytes (total 16384),
[Reader] read 4096 bytes (total 20480),
[Reader] read 4096 bytes (total 24576),
[Reader] read 4096 bytes (total 28672),
[Reader] read 4096 bytes (total 32768),
[Reader] read 4096 bytes (total 36864),
[Reader] read 4096 bytes (total 40960),
[Reader] read 4096 bytes (total 45056),
[Reader] read 4096 bytes (total 49152),
[Reader] read 4096 bytes (total 53248),
[Reader] read 4096 bytes (total 57344),
[Reader] read 4096 bytes (total 61440),
[Reader] read 4096 bytes (total 65536),
[Reader] read 4096 bytes (total 69632),
[Reader] read 4096 bytes (total 73728),
[Reader] read 4096 bytes (total 77824),
[Reader] read 4096 bytes (total 81920),
[Reader] read 4096 bytes (total 86016),
[Reader] read 4096 bytes (total 90112),
[Reader] read 4096 bytes (total 94208),
[Reader] read 4096 bytes (total 98304),
[Reader] read 4096 bytes (total 102400),
[Reader] read 4096 bytes (total 106496),
[Reader] read 4096 bytes (total 110592),
[Reader] read 4096 bytes (total 114688),
```

```
[Reader] read 4096 bytes (total 118784),
[Reader] read 4096 bytes (total 122880),
```

We can see that Writer does not write 65535 bytes per time, It write all 122880 bytes (log)
FACT: **write() pass all the loop into Kernel Space**

1. Linux see that Size of written data is bigger than 65535 bytes --> **non-atomic**
2. OS pour full 65535 bytes into Pipe buffer
3. Instead return User-Space and notify "Full 65535 bytes", Write() go to sleep state inside **Kernel-Space**
4. Reader will read 4096 bytes per time, then Kernel wake up Write() and pour 4096 bytes into Pipe. Loop again
5. This loop is handle inside **Kernel , not User-Space**

1.3 popen() pclose()

popen() = pipe() + fork() + dup2() + exec()
pclose() = waitpid(pid1,...), waitpid(pid2, ...)

We have an example convert Code1 **use** pipe() + fork() + dup2() + exec() + waitpid()
to Code 2 **use** only popen() and pclose()

This example will illustrate `ls | wc -l` shell command

Code1:

```
int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {     /*Create pipe*/
        perror("Failure on create pipe");
        exit(1);
    }
    pid1 = fork();
    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO); /* Connect stdout of this process to the write of pipe
*/

        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);
        close(pipefd[1]);
```

```
        execlp("ls", "ls", NULL); /* Overwrite by execlp */
        perror("exec fail");
        exit(1);
    }
    pid2 = fork();
    if (pid2 == 0) {
        dup2(pipefd[0], STDIN_FILENO); /* Connect the read end of pipe to stdin of this
process*/

        /*Copied success, close head and tail of pipe*/
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL); /* Overwrite by execlp */
        perror("exec fail");
        exit(1);
    }
    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
```

Code2:

```
    FILE *fp;
    char buffer[128];

    /* popen do: pipe() + fork() + dup2() + execlp("/bin/sh","-c","ls | wc -l") */
    fp = popen("ls | wc -l", "r");
    if (fp == NULL) {
        perror("popen failed");
        exit(1);
    }

    /* Get the result from FILE fp (return type of popen) */
    if (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
```

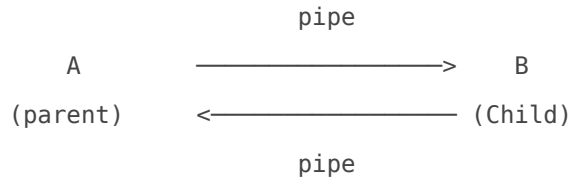
```
    int status = pclose(fp);
    if (status == -1) {
        perror("pclose failed");
        exit(1);
    }

    return 0;
```

Compare 2 methods:

Code 1	Code 2
pipe(fd)	Shell creates internal pipe between `ls` and `wc -l`
1st fork() for `ls`, 2nd fork() for `wc`	Shell self-fork() 2 processes for `ls` and `wc -l`
dup2(pipefd[1], STDOUT_FILENO) in child1	Shell self-bind stdout of `ls` into pipe
dup2(pipefd[0], STDIN_FILENO) in child2	Shell self-bind stdin of `wc -l` into pipe
close(pipefd[0]) / close(pipefd[1])	Shell self-manage
execlp("ls",...), execlp("wc","-l",...)	Merge in to a string "ls \ wc -l"
waitpid(pid1,...), waitpid(pid2,...)	pclose(fd) to self-wait shell child process terminate

1.3 Coprocesses




```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
int main() {
int to_child[2];    // A write -> B đọc (dup2 into stdin of B)
int from_child[2];  // B write -> A đọc (dup2 into stdout of B)

pipe(to_child);
pipe(from_child);
```

```
pid_t pid = fork();
if (pid == 0) {
    // Process B (coprocess)
    dup2(to_child[0], STDIN_FILENO);
    dup2(from_child[1], STDOUT_FILENO);

    close(to_child[0]); close(to_child[1]);
    close(from_child[0]); close(from_child[1]);

    execlp("some_filter", "some_filter", NULL);
    exit(1);
}

// Process A (parent)
close(to_child[0]);      // A does not read this pipe
close(from_child[1]);    // A dose not write on this pipe

write(to_child[1], "input data\n", 11);   // Assump send data to B
char buf[128];
read(from_child[0], buf, sizeof(buf));    // B received
}
```

In summary:

- Pipe or anonymous Pipe, has no name and no exist by **physical file on disk**. It's temporary buffer locate in **kernel RAM**
  - Pipe use FIFO mechanism
  - Only use with relative relationship process (parent - child; child - child (in 1 process)

1.4 FIFO (Named Pipe)

Overview:

- FIFO has a name within file system
- So that, FIFO can communicate between unrelated processes
- As with pipe, when all descriptors reffering to a FIFO have been closed, any outstanding data is discarded

2 ways to create a FIFO

```
$ mkfifo [ -m mode ] pathname     /* On shell */
-----------------------------------------------
#include <sys/stat.h>
int mkfifo(const char* pathname, mode_t mode);       /* Function */
                 /* Return 0 on success, -1 on error */
```

mode = **O_RDONLY, O_WRONLY, O_RDWR**

Synchonization mechanism:

- If process A open FIFO with Write only Mode, `open()` block until process B open FIFO with Read Only Mode
- Both Read and Write Mode appear, then `open()` is unblock and continue run

Using FIFO to design a Client-Server Application

```
/* server.c */

#define FIFO_REQUEST  "/tmp/fifo_request"
#define FIFO_RESPONSE "/tmp/fifo_response"

int main() {
    /* Create 2 FIFO */         /* mode: permission mask */
    mkfifo(FIFO_REQUEST, 0666);  // 1st char: 0 (octal) not Decimal, 2nd char: User/Owner
(create file)
    mkfifo(FIFO_RESPONSE, 0666); // 3rd char: Group (same level with Owner), Last char: Others
(Any user)
                                 // 6 = 4 (read)  + 2 (write)

    printf("[Server] Wait for client connect...\n");

    while (1) {
        int req_fd = open(FIFO_REQUEST, O_RDONLY);
        if (req_fd == -1) { perror("open request"); continue; }

        char buffer[256];
        ssize_t n = read(req_fd, buffer, sizeof(buffer) - 1);
        close(req_fd);   /* close after read successfully 1 request, open in next loop*/

        if (n <= 0) continue;
        buffer[n] = '\0';
```

```
        printf("[Server] Received from client: %s\n", buffer);

        /* Convert to Upper */
        for (int i = 0; buffer[i]; i++)
            buffer[i] = toupper(buffer[i]);

        /* Open FIFO response to send result to client */
        int resp_fd = open(FIFO_RESPONSE, O_WRONLY);
        if (resp_fd == -1) { perror("open response"); continue; }
        write(resp_fd, buffer, strlen(buffer));
        close(resp_fd);
    }
    return 0;
}
```

```
/* client.c */
#define FIFO_REQUEST  "/tmp/fifo_request"
#define FIFO_RESPONSE "/tmp/fifo_response"

int main() {
    char msg[256];
    printf("Write message to send to server: ");
    fgets(msg, sizeof(msg), stdin);
    msg[strcspn(msg, "\n")] = '\0';   /* dont use newline char */

    /* Send request */
    int req_fd = open(FIFO_REQUEST, O_WRONLY);
    if (req_fd == -1) { perror("open request"); exit(1); }
    write(req_fd, msg, strlen(msg));
    close(req_fd);

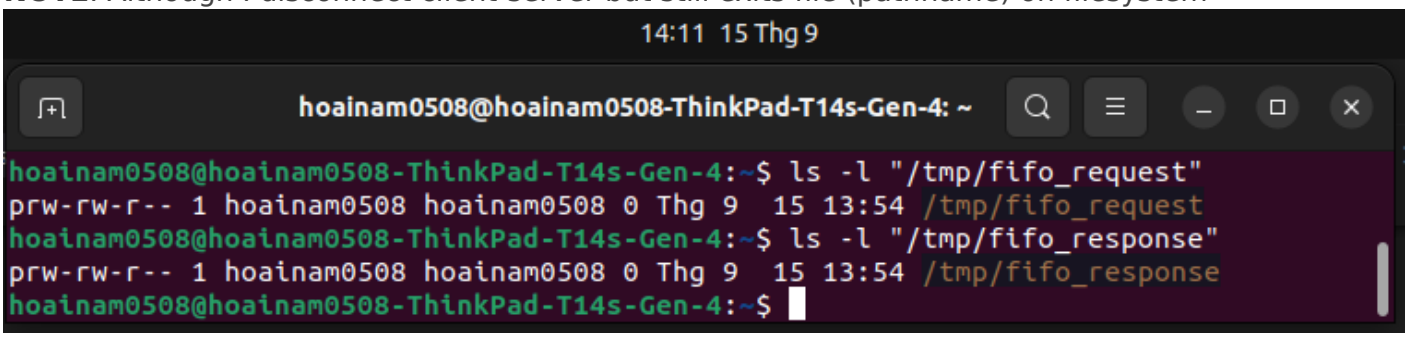
    /* Wait và read response */
    int resp_fd = open(FIFO_RESPONSE, O_RDONLY);
    if (resp_fd == -1) { perror("open response"); exit(1); }

    char buffer[256];
    ssize_t n = read(resp_fd, buffer, sizeof(buffer) - 1);
    close(resp_fd);

    if (n > 0) {
```

```
        buffer[n] = '\0';
        printf("[Client] Server answer: %s\n", buffer);
    }
    return 0;
}
```

NOTE: Although I disconnect client-server but still exits file (pathname) on filesystem



Although permit 0666 (all read write) but output of `ls -l` is `prw-rw-r`
p: Named Pipe (FIFO)
Linux has protect filter, is **umask**. Reality authority is 0664, not 0666.\ It mean: `Reality authority = 0666 AND NOT 0002 = 0664` (OS will decrease your authority in **umask**)

If we want to terminate this file, use `unlink(const char* pathname)` or `rm` (remove) command
Although file is exits but if we close fd, buffer (file descriptor) is not exits too

2. IPC Identifiers, Keys, Permission, Limits

2.1 IPC Keys and IPC Identifiers

To create or open one object about System V IPC, process have to call `get`
like **(msgget(), semget(), or shmget())**

1. Definition and Relationship

- **Key (key_t)**: integer value, have purpose like name of IPC object
- **Identifier**: integer value is returned after **get()** function, to identify for IPC object is openning at the moment
- **Different**: file descriptor is attribute of each process, but IPC identifier has **system-wide attribute**. Any process know this **identifier** can access directly into IPC object

2. Create unique key

- Use const `IPC_PRIVATE`: example `shmget(IPC_PRIVATE,...)`
- Use `ftok()` (*file to key*) function: `key_t ftok(char* pathname, int proj);`
  - Return integer key on success, -1 on error

Example:

```
int main() {
    key_t key;
    int id;
    key = ftok("/home/hoainam0508/Documents/IPC/ftok.c", 'x');
    if (key == -1)
        exit(1);
    id = msgget(key, IPC_CREAT | S_IRUSR | S_IWUSR);
    if (id == -1)
        exit(1);
    else printf("Create KEY successfully. Key = %d\n", id);

    return 0;
}
```

Result: `Create KEY successfully. Key = 0`
If I change `'x'` to `'A'`, the result is: `Create KEY successfully. Key = 1`
It's a **table** inside kernel. If we call `msgget(x)`,it's first queue (Index 0)
But we call `msgget(A)` after, key x haven't deleted. So key A belong Index 1

We can use `ipcs` to check:

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x780660d3 0          hoainam050 600        0            0
0x410660d3 1          hoainam050 600        0            0
```

0x780660d3 : hex of 'x'
0x410660d3: hex of 'A'

NOTE that: `pathname` in ftok() function is **regular file**, not **system file**

Compare Key (after ftok function) vs Identifier (get function)

If I added `printf("Key after ftok function is %d\n", key);` into above code. The result is

```
Key after ftok function is 2013683923
Create KEY successfully. Key = 0
```

Characteristic	Key	Identifier
Meaning	External/Global Name	Internal Identifier
Purpose	Dependent Processes can find each other	Process can operate directly to resource
Understand	like a name of IPC	Represent to IPC is open

3. Algorithm to create **identifier** of Kernel

- Mathematical: `identifer = index + seq * SEQ_MULTIPLIER` (index is the position in data page of kernel, SEQ_MULTIPLIER = 32768, Seq = sequence number)
- **PURPOSE**: if server die and restart, old IPC object is deleted and new IPC is created in **same index** position so *seq* increase, then new **identifier** differ old

2.2 IPC permission

Kernel manage Permission of each IPC object through `ipc_perm` component in kernel (including **uid, gid, cuid, cgid** and bitmask **mode**) (user, group, creator user, creator group)

- Only have Read and Write authority
- Check authority: Kernel check by **Effective User ID (EUID), Effective Group ID (EGID)** and **Supplementary Group IDs** of calling processes Note: Linux check UID/GID; but IPC check Effective UID/GID
- To delete or change attribute of IPC:
  - Read or write, use Read/Write authority
  - To delete **IPC_RMID** or change structure **IPC_SET**, Effective UID = **Owner UID, Creator UID**. Or process must have authority: **CAP_SYS_ADMIN, CAP_IPC_OWNER**

```
struct ipc_perm {
key_t __key;      /* Key, as supplied to 'get' call */
uid_t uid;        /* Owner's user ID */
gid_t gid;        /* Owner's group ID */
uid_t cuid;       /* Creator's user ID */
gid_t cgid;       /* Creator's group ID */
unsigned short mode;   /* Permissions */
```

```
unsigned short __seq;  /* Sequence number */
};
```

2.3 IPC Limits

Run `ipcs -l`

```
------ Messages Limits --------
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384

------ Shared Memory Limits --------
max number of segments = 4096
max seg size (kbytes) = 18014398509465599     /* 16 EB (Exabytes) = 16 * 1024 PB (Peta) = 16 *
1024^2 TB */
max total shared memory (kbytes) = 18446744073709551612
min seg size (bytes) = 1

------ Semaphore Limits --------
max number of arrays = 32000
max semaphores per array = 32000
max semaphores system wide = 1024000000
max ops per semop call = 500
semaphore max value = 32767
```

3. Message Queue (System V)

Overview: message queue is data exchange method belong **message** in **data tranfer** mechanism. The characteristics is **limited data exchange**

The different from pipes and FIFO:

- Handle through *identifier* returned by msgget()

- Communication is **oriented** --> reader receive whole messages, as written by writer
  - not possible to read part of message, leaving the remainder in queue, or read multiple message at a time
  - **contrast** with pipes: provide an undifferentiated stream of bytes
- To contain data, each *message* has integer `type`.
- Message can be **retrieved** from a queue in first-in first-out order or **retrieved** by `type`

3.1 Create or open message queue

```
int main() {
    key_t key;
    int id;
    key = ftok("/home/hoainam0508/Documents/IPC/ftok.c", 'B');
    if (key == -1)
        exit(1);
    else printf("Key after ftok function is %d\n", key);
    id = msgget(key, IPC_CREAT | IPC_EXCL);
    if (id == -1) {
        printf("Can not create or open ms\n");
        exit(1);
    }
    else printf("Create KEY successfully. Key = %d\n", id);

    return 0;
}
```

I create Key 'x' first, then delete it by `ipcrm -q` command. Then create Key 'x' again & Key "A" (both normal) and Key 'B' (Exclusive flag) So the result of `ipcs -q`:

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0                   /* Key x */
0x780660d3 2          hoainam050 600        0            0                   /* Key A */
0x420660d3 3          hoainam050 0          0            0                   /* Key B */
exclusive
```

Why 0?
0 = 000 (no authorize access for **Owner, Group and Others User**)
--> to have authority, pass authotiry value in to `msgget()` function

```
key = ftok("/home/hoainam0508/Documents/IPC/ftok.c", 'C');     /* Key C */
id = msgget(key, 0666 | IPC_CREAT | IPC_EXCL);
```

So the result:

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
0x430660d3 4          hoainam050 666        0            0         /* Key C */ is just created
```

3.2 Exchange Messages

Send message

msgflg: **IPC_NOWAIT** --> instead block when message queue is full, msgsnd() returns immediately with the error **EAGAIN** Prototype:

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
```

Assump we must send message:

```
struct my_msg {
    long mtype;      /* Must be > 0 */
    char text[100];
} msg;
```

- msgid is identifier of message queue
- msgp: type of void, to allow it to be a pointer to any type of structure
- msgz : number of bytes contained in the *text* field
- msgflg: 0 or IPC_NOWAIT

Example:

```
struct my_msg {
    long mtype;
    char text[100];
} msg;
```

```
int main () {
    msg.mtype = 1;
    strcpy(msg.text, "Hello Queue");
    int snd = msgsnd(4, &msg, sizeof(msg) - sizeof(long), 0);
    if (snd == -1) {
        printf("Send Message unsuccess\n");
        exit(1);
    }
    else printf("Send successfully\n");
}
```

Alignment & Padding (Optimization hidden memory Mechanism)

NOTE: Each times We send the message (Example above), the used-byte increase 104 bytes

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
0x430660d3 4          hoainam050 666        912          9

hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ ipcs -q      (After send message)

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
0x430660d3 4          hoainam050 666        1016         10
```

Allign follow multiples (bội số) of 8 bytes (Max size of type in struct)
gcc see 100 bytes (**text[100]**)
but 100 not divisible by 8
--> add 4 bytes (104 divisible by 8)

Received Message

reads (and removes) a message from a message queue, and copies its contents into the buffer pointed to by *msgp*.
Prototype:

```
ssize_t msgrcv(int msqid, void *msgp, size_t maxmsgsz, long msgtyp, int msgflg);
                                      /* Returns number of bytes copied into mtext field, or
–1 on error */
```

- maxmsgsz: The maximum space available in the *text* field . IF no left message for removed, *msgrcv()* return error **E2BIG**. Use MSG_NOERROR flag

We can select message to read according to *mtype* field. Through:

- `msgtyp`
  - msgtyp = 0: removed first message from queue & returned to calling process
  - msgtyp > 0: first message from queue whose `msgtyp` = `mtype` is removed and returned to the calling process
  - msgtyp < 0: treat *waiting message* as a *priority queue*. The first message of the lowest `mtype` less than or equal to the absolute value of `msgtyp` is removed and returned to the calling process.
- `msgflg`: IPC_NOWAIT | MSG_EXCEPT | MSG_NOERROR

Example:

```
struct my_msg {
    long mtype;
    char text[100];
};
void send_message () {
    struct my_msg snd_msg;
    snd_msg.mtype = 1;
    strcpy(snd_msg.text, "Hello Queue");

    int snd = msgsnd(2, &snd_msg, sizeof(snd_msg) - sizeof(long), 0);
    if (snd == -1) {
        printf("Send Message unsuccess\n");
        exit(1);
    }
    else printf("Send successfully\n");
}
void receive_message() {
    struct my_msg rcv_msg;
    ssize_t rcv = msgrcv(2, &rcv_msg, sizeof(rcv_msg) - sizeof(long), 1, IPC_NOWAIT);
```

```
    if (rcv == -1) {
        printf("cannot receive message\n");
        exit(1);
    }
    else {
        printf("Receive/Removed %ld bytes, message: %s\n",sizeof(rcv_msg) - sizeof(long),
rcv_msg.text);
    }
}
int main() {
    // send_message();
    receive_message();
}
```

No message left:

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
0x430660d3 4          hoainam050 666        0            0
```

Block when attempt receive/remove message from queue --> use IPC_NOWAIT flag to return immediately error code
Note: Queue with perms `0` cannot read or write anything message

Control Message

Prototype:

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

- cmd:
  - IPC_RMID: Immediately remove the message queue **object** and its associated msqid_ds data structure
  - IPC_STAT: copy *msqid_ds* data structure
  - IPC_SET: update selected fields of *msqid_ds* using value provided in the buffer pointed to by *buf*

Example: message queue id 4 after use `msgctl` with IPC_RMID:

```
------ Message Queues --------                                          (Before)
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
0x430660d3 4          hoainam050 666        520          5

int ctl = msgctl(4, IPC_RMID, NULL);

------ Message Queues --------                                         (After)
key        msqid      owner      perms      used-bytes   messages
0x410660d3 1          hoainam050 600        0            0
0x780660d3 2          hoainam050 600        0            0
0x420660d3 3          hoainam050 0          0            0
```

Message Queue Associated Data Structure & Limits

```
struct msqid_ds {
struct ipc_perm msg_perm;        /* Ownership and permissions */
time_t            msg_stime;     /* Time of last msgsnd() */
time_t            msg_rtime;     /* Time of last msgrcv() */
time_t            msg_ctime;     /* Time of last change */
unsigned long   __msg_cbytes;    /* Number of bytes in queue */
msgqnum_t         msg_qnum;      /* Number of messages in queue */
msglen_t          msg_qbytes;    /* Maximum bytes in queue */
pid_t             msg_lspid;     /* PID of last msgsnd() */
pid_t             msg_lrpid;     /* PID of last msgrcv() */
};
```

Limits:

```
/proc/sys/kernel$ cat msgmni      /* Number of message queue identifiers that can be created
*/
32000             msgget();
/proc/sys/kernel$ cat msgmax      /* Number of (text) bytes can be written is a single message
*/
8192              msgsnd();
```

```
/proc/sys/kernel$ cat msgmnb      /* Max number of (text) bytes can be held in msq at one time
*/
16384                       /* If limit is reached, msgsnd() blocks or fail with EAGAIN (use
WNOWAIT flag) */
```

Can change MSQ limits

- temporary:
  - echo (overwrite): `sudo sh -c "echo 65536 > /proc/sys/kernel/msgmax"`
  - use `sysctl`: `sudo sysctl -w kernel.msgmax=65536`
- pernament: (2 steps)
  1. Open configure file by ROOT: `sudo nano /etc/sysctl.conf`
  2. Add this line into tail of file: `kernel.msgmax = 65536`

4. Shared Memory (System V)

4.1 Creating or Opening a Shared Memory Segment

Prototype:

```
int shmget(key_t key, size_t size, int shmflg);
                            /* Returns shared memory segment identifier on success, or –1 on
error */
```

- size (> 0):
  - size of segment, in bytes. **Kernel allocates shared memory in multiples (bội số) of the system page size, so *size* is effectively rounded up to the next multiple of the system page size**
  - If existing segment, size has no effect. But it must be **less than or equal** to the size of segment
- shmflg: IPC CREAT | IPC EXCL

Default shared memory segment on my device: `ipcs -m -p`

```
------ Shared Memory Creator/Last-op PIDs --------
shmid      owner      cpid       lpid
6          hoainam050 3266       2677    /* MS Teams for Linux */
7          hoainam050 3266       2677    /* MS Teams for Linux */
41         hoainam050 41400      2677    /* VS Code */
42         hoainam050 41400      2677    /* VS Code */
51         hoainam050 3266       2677    /* MS Teams for Linux */
52         hoainam050 3266       2677    /* MS Teams for Linux */
58         hoainam050 41280      57705   /* VS code */
62         hoainam050 1739       55692   /* chrome */
```

Example:

```
int main() {
    key_t key;
    int id;
    key = ftok("/home/hoainam0508/Documents/IPC/ftok.c", 'C');
    if (key == -1)
        exit(1);
    else printf("Key after ftok function is %d\n", key);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) {
        printf("cannot get identifier\n");
        exit(1);
    }
    else printf("Segment is created: %d\n", shmid);
    return 0;
}
```

After create new segment

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 6          hoainam050 606        504000     2          dest
0x00000000 7          hoainam050 606        504000     2          dest
....        ...        ...       ...         ...        ...        ...
0x00000000 62         hoainam050 600        524288     2          dest
0x430660d3 63         hoainam050 666        1024       0
```

4.2 Using shared memory

Prototype:

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
            /* Returns address at which shared memory is attached on success, or (void *) –1
on error */
```

> attaches the **shared memory segment** identified by *shmid* to the calling process's **virtual address space**

- *shmaddr*
  - NULL: segment is attached at a suitable address selected by the kernel. **It's preferred method**
  - not NULL, SHM_RND not set: segment is attached at the address specified by *shmaddr*, which must be **multiple of the system page size**
  - not NULL, SHM_RND is set: segment is mapped at the address provided in *shmaddr*, rounded down to the nearest multiple of the constant SHMLBA (*shared memory low boundary address_*) --> improve CPU cache performance on some architectures

Specifying a **non-NULL** value for *shmaddr is* **not recommended**, because:

- reduces the portability of an application. An address valid on one UNIX implementation may be invalid on another.
- may be address (specified) in use

Value	Description
SHM_RDONLY	Attach segment read-only
SHM_REMAP	Replace any existing mapping at shmaddr
SHM_RND	Round shmaddr down to multiple of SHMLBA bytes
Example:	

```
/* write.c */
void write_data() {
    char *str = (char *) shmat(63, NULL, 0);
    if (str == (void*) - 1) {
        exit(1);
    }
    strcpy(str, "Hello Shared Memory");
```

```
    printf("Data written: %s\n", str);
}
int main() {
    write_data();
}
```

When a process no longer needs to access a shared memory segment

```
int shmdt(const void *shmaddr);            /* Detach segment from its virtual address space
*/
                                           /* Returns 0 on success, or –1 on error */
```

NOTE THAT: **Detach is not Delete**

- exec() in child after fork() = detach
- terminated a process = detach

Detach	Remove
Return pointer, dont want to see memory	Require Kernel delete memory
Local (Calling process)	Global (Effect on resource of System-Wide)
Memory, data are exist on RAM of kernel	Free all data, RAM is clean

Example:

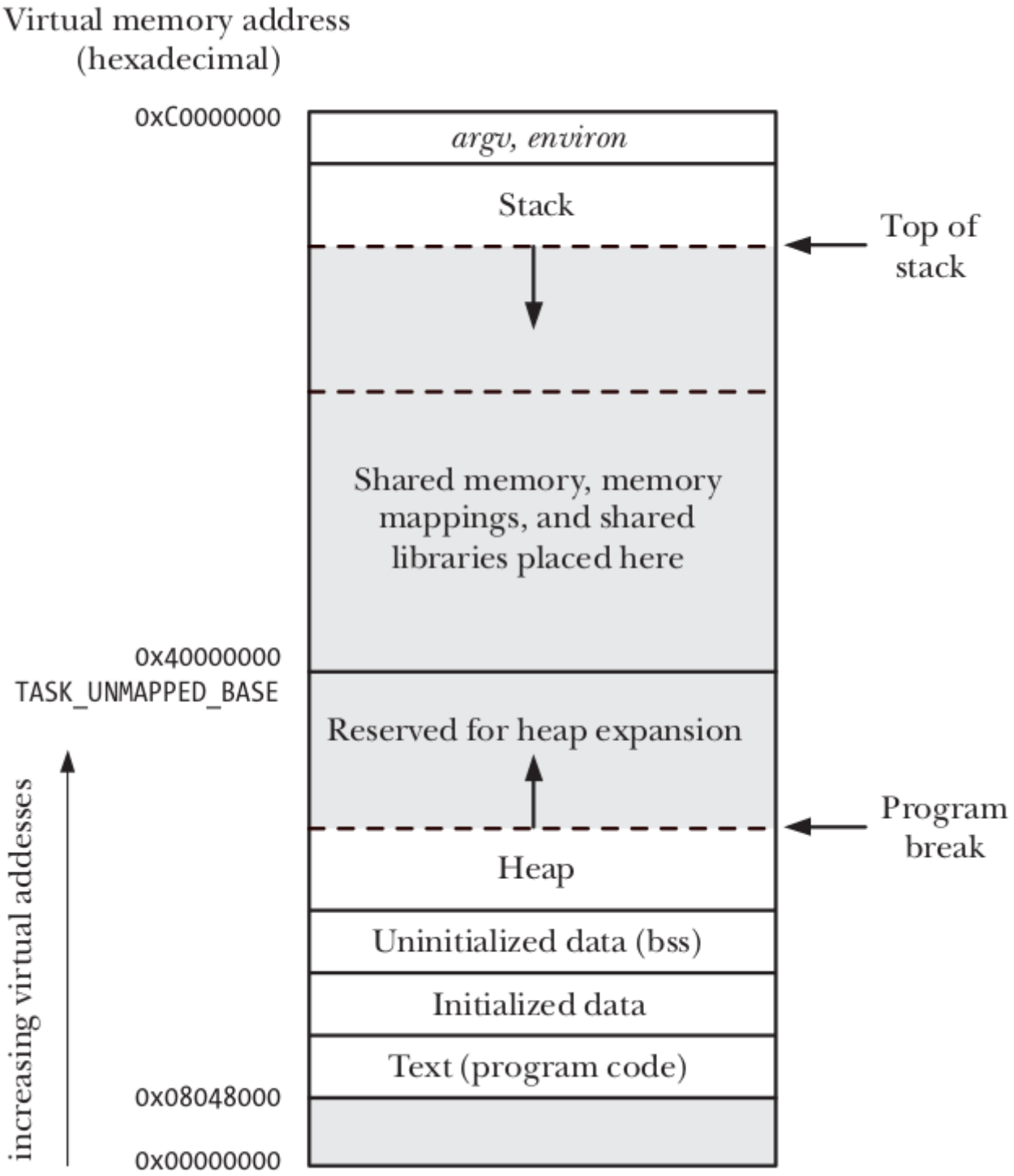
```
/* read_data.c */
void write_data() {
    char *str = (char *) shmat(63, NULL, 0);
    if (str == (void*) - 1) {
        exit(1);
    }
    strcpy(str, "Hello Shared Memory");
    printf("Data written: %s\n", str);
}
int main() {
    write_data();
}
```

IN SUMMARY:

- shmat() create new segment in **page table** of this calling process, Point to a physical memory where is allocated by kernel through `shmget()` function

- virtual address may be different from processes
- but **physical memory** is unique (only one)
- A write, B read immediately (need no copy to kernel like **message**) because both point to **physical memory**

4.3 Location of Shared Memory in Virtual Memory



We can see the location of the shared mem segments and shared lib mapped by a program: `cat /proc/PID/maps`

Run `cat /proc/62797/maps` PID of ./write_data

```
virtual space range | Protection & flag | offset | device number (major:minor IDs) |
identifier of segment|.
/* main program, correspond to the text and data segments of the program */
59c752fc8000-59c752fc9000 r--p 00000000 103:06 2755749
```

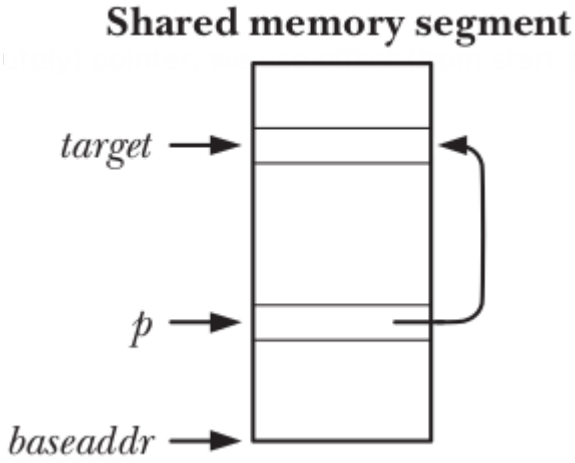
```
/home/Documents/IPC/shared_mem/write_data
59c752fc9000-59c752fca000 r-xp 00001000 103:06 2755749
/home/Documents/IPC/shared_mem/write_data
59c752fca000-59c752fcb000 r--p 00002000 103:06 2755749
/home/Documents/IPC/shared_mem/write_data
59c752fcb000-59c752fcc000 r--p 00002000 103:06 2755749
/home/Documents/IPC/shared_mem/write_data
59c752fcc000-59c752fcd000 rw-p 00003000 103:06 2755749
/home/Documents/IPC/shared_mem/write_data
/* ---------------------------------------------------------------- */
59c781c60000-59c781c81000 rw-p 00000000 00:00 0                          [heap]
7b8f15bfd000-7b8f15c00000 rw-p 00000000 00:00 0                             --> anonymous
mapping
7b8f15c00000-7b8f15c28000 r--p 00000000 103:05 132776   /usr/.../libc.so.6
7b8f15c28000-7b8f15dbd000 r-xp 00028000 103:05 132776   /usr/.../libc.so.6
7b8f15dbd000-7b8f15e15000 r--p 001bd000 103:05 132776   /usr/.../libc.so.6     --> Shared
library (.so / .a)
7b8f15e15000-7b8f15e16000 ---p 00215000 103:05 132776   /usr/.../libc.so.6     --> standard C
lib
7b8f15e16000-7b8f15e1a000 r--p 00215000 103:05 132776   /usr/.../libc.so.6
7b8f15e1a000-7b8f15e1c000 rw-p 00219000 103:05 132776   /usr/.../libc.so.6
7b8f15e1c000-7b8f15e29000 rw-p 00000000 00:00 0
7b8f15e3a000-7b8f15e3c000 rw-p 00000000 00:00 0
7b8f15e3c000-7b8f15e3e000 r--p 00000000 103:05 132761   /usr/.../ld-linux-x86-64.so.2 -->
dynamic linker
7b8f15e3e000-7b8f15e68000 r-xp 00002000 103:05 132761   /usr/.../ld-linux-x86-64.so.2
7b8f15e68000-7b8f15e73000 r--p 0002c000 103:05 132761   /usr/.../ld-linux-x86-64.so.2
7b8f15e73000-7b8f15e74000 rw-s 00000000 00:01 63        /SYSV430660d3 (deleted)       --> SysV
shared memory
7b8f15e74000-7b8f15e76000 r--p 00037000 103:05 132761   /usr/.../ld-linux-x86-64.so.2
7b8f15e76000-7b8f15e78000 rw-p 00039000 103:05 132761   /usr/.../ld-linux-x86-64.so.2
7fff17f4f000-7fff17f71000 rw-p 00000000 00:00 0                          [stack]
7fff17fec000-7fff17ff0000 r--p 00000000 00:00 0                          [vvar]
7fff17ff0000-7fff17ff2000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

4.4 Storing Pointers in Shared Memory

PROBLEM: When 2 or more processes attach shared memory segment into their virtual space memory. Each process has attach this segment into different virtual memory (**by kernel**)
When we use normal C pointer (**void* ptr**):

- Can use this poniter in only calling process
- When another process access to this virtual space memory, they cannot hold the right virtual address of pointer

Solution: Instead use (absol ddress of shared mem segment to pointer address)



Wrong: *p = target
Right: *p = (target - baseaddr) --> Dereference pointer: target = baseadd + *p;

Im summary: Offset between elements in shared memory segment is not change when acesseed from different process

4.5 Shared Memory Control Operations

Prototype:

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
                                                /* Returns 0 on success, or –1 on error */
```

Generic control operations

- IPC_RMID: Delete shared mem segment and its data structure . **The segment is removed after all processes have detached from it**
  - If processes dont detach this segment, the result:
    - status = dest (destroy)
    - key = 0 (0x00000000) : IPC_PRIVATE
    - Cannot attach `shmat()` through old key
    - With processes haven't detach: no effect (can be read/write inside segment)
  - Only delete/free segment when the last process detach this segment `nattch = 0` --> Kernel free this physical mem

Lock and unlock Shared Memory Linux has **Swapping** mechanism, its mean whenever shortage RAM from system, it will be move unnecessary data RAM to disk (for another task). --> We use SHM_LOCK to locked into RAM, so that it is never swapped out

- The SHM_LOCK operation locks a shared memory segment into memory.
- The SHM_UNLOCK operation unlocks the shared memory segment, allowing it to be swapped out.

4.6 Shared Memory Associated Data Structure & Limits

```
struct shmid_ds {
struct ipc_perm shm_perm;  /* Ownership and permissions */
size_t shm_segsz;          /* Size of segment in bytes */
time_t shm_atime;          /* Time of last shmat() */
time_t shm_dtime;          /* Time of last shmdt() */
time_t shm_ctime;          /* Time of last change */
pid_t  shm_cpid;           /* PID of creator */
pid_t  shm_lpid;           /* PID of last shmat() / shmdt() */
shmatt_t shm_nattch;       /* Number of currently attached processes */
};
```

The limits:

```
cat shmmni       /* number of shared memory identifiers */
4096
cat shmmax        /* maximum size (in bytes) of a shared memory segment */
18446744073692774399
cat shmall        /* total number of pages of shared memory */
```

```
18446744073692774399
```

Can change this limit like Message Queue

5. Semaphore (System V)

Create or Openning a Semaphore Set

```
int semget(key_t key, int nsems, int semflg);
```

- nsems specifies the number of semaphores in that set, and must be greater than 0
- If we are using semget() to obtain the identifier of an existing set, then nsems must be less than or equal to the size of the set (or the error EINVAL results)
- It is not possible to change the number of semaphores in an existing set.

Semaphore Control Operations

```
int semctl(int semid, int semnum, int cmd, ... /* union semun arg */);
                           /* Returns nonnegative integer on success (see text); returns –1
on error */
```

- *semnum* argument identifies a particular semaphore within the set
- The cmd argument specifies the operation to be performed.
  - IPC_RMID: Immediately remove the semaphore set and its associated semid_ds data structure

Semaphore Associated Data Structure

```
struct semid_ds {
struct ipc_perm sem_perm;     /* Ownership and permissions */
time_t sem_otime;             /* Time of last semop() */
time_t sem_ctime;             /* Time of last change */
unsigned long sem_nsems;      /* Number of semaphores in set */
};
```

Semaphore Limit

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:/proc/sys/kernel$ ipcs -s -l


------ Semaphore Limits --------
```

```
max number of arrays = 32000
max semaphores per array = 32000
max semaphores system wide = 1024000000
max ops per semop call = 500
semaphore max value = 32767
```

6. POSIX - Message Queue

6.1 Open, Close, Unlink Message Queue

To creates a new message queue or opens an existing queue

```
mqd_t mq_open(const char *name, int oflag, mode_t mode, struct mq_attr *attr );
                            /* Returns a message queue descriptor on success, or (mqd_t) –1
on error */
```

- name: pathname from temporary system file
- oflag: bit mask, control the operation of *mq_open()*

Flag	Description
O_CREAT	Create queue if it doesn't already exist
O_EXCL	With O_CREAT, create queue exclusively
O_RDONLY	Open for reading only
O_WRONLY	Open for writing only
O_RDWR	Open for reading and writing
O_NONBLOCK	Open in nonblocking mode

If existing message queue, the call requires only two arguments (1st and 2nd arguments)
However, if O_CREAT is specified in flags, two further arguments are required: *mode* and *attr*

- mode: bit mask that specifies the permissions to be placed on the new message queue
- attr: is an mq_attr structure that specifies attributes for the new message queue Example:

```
struct mq_attr attr;
int main() {
    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;      // maximum 10 messages in queue
    attr.mq_msgsize = 256;    // maximum size per message
    mqd_t mq = mq_open("/my_queue", O_CREAT, 0644, &attr);
    if (mq == -1) {
        printf("Error create message\n");
        exit(1);
    }
    else printf("Create message queue Successfully\n");

    return 0;
}
```

Result: `Create successfully`
We can check this file on filesystem:

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ ls /dev/mqueue/my_queue
/dev/mqueue/my_queue
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ cat /dev/mqueue/my_queue
QSIZE:0          NOTIFY:0     SIGNO:0     NOTIFY_PID:0
```

Another process:

```
int main() {
    mqd_t mq2 = mq_open("/my_queue", O_RDONLY);
    return 0;
}
```

Result: `Read successfully`

Closing a message queue
Example:

```
int main() {
    mqd_t mq2 = mq_open("/my_queue", O_RDONLY);
    int close = mq_close(mq2);
    if (close == -1) {
        printf("Cant close msq\n");
        exit(1);
```

```
    }
    else printf("Closed msq success\n");
    return 0;
}
```

We can close msq from process which is not creator **UNLINK**

```
  mqd_t mq2 = mq_open("/my_queue", O_RDONLY);


    int unlink = mq_unlink("/my_queue");
    if (unlink == -1) {
        printf("Cant unlink msq\n");
        exit(1);
    }
    else printf("Unlink msq success\n");
```

After unlink: `ls /dev/mqueue/my_queue`
Result: `ls: cannot access '/dev/mqueue/my_queue': No such file or directory`
--> Deleted message queue

Relationship Between Descriptors and Message Queue

6.2 Exchange messages

adds the message in the buffer pointed to by *msg_ptr* to the message queue referred to by the descriptor *mqdes*
Prototype:

```
#include <mqueue.h>
int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
                                                        /* Returns 0 on success, or –1 on
error */
```

- mqdes: value is returned by `mq_open()`
- msg_len: length of the message pointed to by msg_ptr
  - must be less than or equal to the *mq_msgsize*
  - otherwise, mq_send() fails with the error **EMSGSIZE**
  - Zero-length messages are permitted
- msg_prio: Messages are ordered within the queue in descending order of priority
  - 0 is the lowest
  - if has new message, it's placed after any other languages of the same priority

- dont use priority = 0.

Example:

```
struct mq_attr attr;
int main() {
    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;      // maximum 10 messages in queue
    attr.mq_msgsize = 256;    // maximum size per message
    attr.mq_curmsgs = 0;

    mqd_t mq = mq_open("/my_queue", O_CREAT | O_WRONLY, 0644, &attr);
    if (mq == -1) {
        printf("Error create message\n");
        exit(1);
    }
    else printf("Create message queue Successfully\n");

    char buf[256] = "Hello from sender";
    int send = mq_send(mq, buf, strlen(buf) + 1, 1);
    if (send == -1) {
        printf("Error send message\n");
        exit(1);
    }
    else printf("Send message Successfully\n");

    printf("Sent: %s\n", buf);
    mq_close(mq);
    return 0;
}
```

Result:

```
Create message queue Successfully
Send message Successfully
Send: Hello from sender
```

NOTE: `msg_len = strlen(buf) + 1` to get `\0` char --> char end of string
After send 18 bytes "Hello from sender" 4 times\

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ cat /dev/mqueue/my_queue
QSIZE:72         NOTIFY:0     SIGNO:0     NOTIFY_PID:0
```

Receive Message:
remove the oldest message with the highest priority from message queue (referred to by *mqdes*), returns the message in the buffer pointed to by *msg_ptr*
Prototype:

```
ssize_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
                                /* Returns number of bytes in received message on success, or –1
on error */
```

- msg_len: specify the number of bytes of space available in the buffer pointed to by msg_ptr.
  - must be greater than or equal to the *mq_msgsize* attribute of the queue
  - otherwise, mq_receive() fails with the error EMSGSIZE
  - if dont know *mq_msgsize*, use *mq_getattr()* to get actual size of message of queue
- msg_prio:
  - priority of received message is copied into the location pointed to by *msg_prio*
  - **IF message queue EMPTY**, mq_receive() blocks until a message becomes available. **OR** use O_NONBLOCK flag, immediately return error EAGAIN

Example:

```
int main() {
    mqd_t mq2 = mq_open("/my_queue", O_RDONLY);
    if (mq2 == -1) {
        printf("Error open message\n");
        exit(1);
    }
    else printf("Open message queue Successfully\n");

    struct mq_attr attr;
    mq_getattr(mq2, &attr);   /* Get actual message size of queue */

    char *buf = malloc(attr.mq_msgsize);
    unsigned int prio;

    ssize_t bytes = mq_receive(mq2, buf, attr.mq_msgsize, &prio);
    if (bytes == -1) {
        printf("Cant receive message\n");
        exit(1);
```

```
    }
    printf("Received (%ld bytes, priority %u): %s\n", bytes, prio, buf);

    free(buf);
    mq_close(mq2);

    return 0;
}
```

Result:

```
Open message queue Successfully
Received (18 bytes, priority 1): Hello from sender
```

The new size: (After receive/get 18 bytes)

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ cat /dev/mqueue/my_queue
QSIZE:54         NOTIFY:0     SIGNO:0     NOTIFY_PID:0
```

Sending and Receiving Messages with a Timeout

- mq_timedsend(): add *abs_timeout argument when message queue is FULL, so that it's wait for RECEIVER can Receive/Get message from queue in fixed time
- mq_timedreceive(): add *abs_timeout argument to wait for Sender send message to queue (Empty queue Case)

6.3 Message Notification (New Feature when Compare with SysV)

Overview:

- Notify when queue transitions from being empty to nonempty (0 ms --> 1 ms in queue)
- Instead block on `mq_receive()` call or marking descriptor nonblocking and perform periodic `mq_receive()` call **polls**, a process can request a notification of message arrival and perform other tasks until it is notified
- can choose notified via a signal or via invocation of a function in a separate thread

calling process will receive a notification when a message arrives on the empty queue

```
int mq_notify(mqd_t mqdes, const struct sigevent *notification);
                            /* 0 succese, -1 error */
```

A few points about message notification:

- can register to receive notification in anytime. If it's already registered, further attemps to register will error (EBUSY)
- The registered process is notified only if some other process is not currently blocked in a call to mq_receive() for the queue. If some other process is blocked in mq_receive(), that process will read the message, and the registered process will remain registered.
- Can deregister if use *notification* = NULL

notification argument:

- SIGEV_NONE: register but no notify when message arrived
- SIGEV_SIGNAL: kernel send signal, specified in the *sigev_signo* field
- SIGEV_THREAD: kernel create new thread, run callback function *sigev_notify_function*

Notify via a Signal

Example:

```
int main() {
    mqd_t mq = mq_open("/my_queue", O_RDONLY);
    if (mq == -1) {
        printf("Error open message\n");
        exit(1);
    }
    else printf("Open message queue Successfully\n");

    struct sigevent sev;
    sev.sigev_notify = SIGEV_SIGNAL;
    sev.sigev_signo = NOTIFY_SIG;

    int ntf = mq_notify(mq, &sev);
    if (ntf == -1) {
        printf("Cannot register notification\n");
        exit(1);
    }
    printf("Waiting for notify...\n");

    while (1) {
```

```
        pause();   /* No cost CPU, wake up by SIGNAL */
    }
}
```

Result:

```
Open message queue Successfully
Waiting for notify...
```

After send 1 message to message queue

```
Open message queue Successfully
Waiting for notify...
User defined signal 1
```

Notify via a Thread

Example:

```
mqd_t mq;
void register_notify();
void thread_func(union sigval sv) {
    char buf[256];
    ssize_t bytes = mq_receive(mq, buf, sizeof(buf), NULL);
    if (bytes >= 0) {
        printf(">> [Thread %lu] Received: %s\n", pthread_self(), buf);
    }

    register_notify();
}
void register_notify() {
struct sigevent sev;
    sev.sigev_notify = SIGEV_THREAD;
    sev.sigev_notify_function = thread_func;
    sev.sigev_notify_attributes = NULL;
    sev.sigev_value.sival_ptr = NULL;

    int ntf = mq_notify(mq, &sev);
    if (ntf == -1) {
        printf("Cannot register notification\n");
```

```
        exit(1);
    }
}
int main() {
    mq = mq_open("/my_queue", O_RDONLY);
    if (mq == -1) {
        printf("Error open message\n");
        exit(1);
    }
    else printf("Open message queue Successfully\n");

    register_notify();
    printf("Main thread is free, no block. Wait for notification create thread...\n");
    while (1) {
        pause();   /* No cost CPU, wake up by SIGNAL */
    }
}
```

Result: (After receive a message)

```
Open message queue Successfully
Main thread is free, no block. Wait for notification create thread...
>> [Thread 127584750794304] Received: Hello from sender
```

The pros of **NOTIFY_BY_SIGNAL**:

- avoid async-signal-safety problem

6.4 Message Queue Attributes && Limit

```
struct mq_attr {
long mq_flags; /* Message queue description flags: 0 or O_NONBLOCK [mq_getattr(),
mq_setattr()] */
long mq_maxmsg; /* Maximum number of messages on queue [mq_open(), mq_getattr()] */
long mq_msgsize; /* Maximum message size (in bytes) [mq_open(), mq_getattr()] */
long mq_curmsgs; /* Number of messages currently in queue [mq_getattr()] */
};
```

Limits

```
/proc/sys/fs/mqueue$ cat msg_max   /* Max message on a queue */
10
/proc/sys/fs/mqueue$ cat msgsize_max  /* Max size of a message (bytes) */
8192
/proc/sys/fs/mqueue$ cat queues_max  /* Max number of queue */
256
```

7. Memory mappings

Overview: `mmap()` creates a new memory mapping in the calling process's virtual address space. 2 types:

- File mapping: A file mapping maps a region of a file directly into the calling process's virtual memory
- Anonymous mapping: An anonymous mapping doesn't have a corresponding file. Instead, the pages of the mapping are initialized to 0.

The memory in one process's mapping may be shared with mappings in other processes. 2 ways:

- two processes map the same region of a file, shared the same pages of physical memory
- a child created by fork() inherits copies of its parent's mappings

When two or more processes share the same pages, each process can see the changes to the page contents made by other processes, depending on whether the mapping is **private** or **shared**

- Private mapping (MAP_PRIVATE): one process modify a content --> not visible to other processes. Using the copy-on-write technique with page table of processes
  --> MAP_PRIVATE is private, copy-on-write mapping
- Shared mapping (MAP_SHARED): Modifications to the contents of the mapping are visible to other processes that share the same mapping and, for a file mapping, are carried through to the underlying file

7.1 Creating a Mapping: mmap()

creates a new mapping in the calling process's virtual address space

```
#include <sys/mman.h>
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

```
                        /* Returns starting address of mapping on success, or MAP_FAILED on
error */
```

- addr:
  - NULL : preferred way, kernel chooses a suitable address
  - non-NULL: kernel takes as a hint about the address
- length: the bytes size of the mapping
  - kernel creates mapping in unit of **multiple of the system page size**
- prot: protection to be placed on the mapping
  - PROT_NONE: The region may not be accesed
  - PROT_READ: contents of region can be read
  - PROT_WRITE: contents of region can be modified
  - PROT_EXEC: contents of region can be executed
- flag: bit mask
  - MAP_PRIVATE
  - MAP_SHARED
- fd: file descriptor identifying the file to be mapped
- offset: the starting point of the mapping in the file

if MAP_FIXED is specified into *prot*, implementation may require that addr be page-aligned

Example:

```
int main() {
    int fd = open("test.txt", O_RDONLY);

    void *mapped = mmap(NULL, 4096, PROT_READ, MAP_PRIVATE, fd, 0);

    if (mapped == MAP_FAILED) {
        printf("Map fail\n");
        return 1;
    }

    printf("Mapping successfully at virtual address: %p\n", mapped);
    while (1) {
        pause();
    }
    // munmap(mapped, 4096);
    close(fd);
    return 0;
}
```

Result: `Mapping successfully at virtual address: 0x7b3a7aeb4000` Check: `cat /proc/78556/maps`

```
5f2fbfa3a000-5f2fbfa3b000 r--p 00000000 103:06 2756768
/home//IPC/mmap/create_a_map
5f2fbfa3b000-5f2fbfa3c000 r-xp 00001000 103:06 2756768
/home//IPC/mmap/create_a_map
5f2fbfa3c000-5f2fbfa3d000 r--p 00002000 103:06 2756768
/home//IPC/mmap/create_a_map
5f2fbfa3d000-5f2fbfa3e000 r--p 00002000 103:06 2756768
/home//IPC/mmap/create_a_map
5f2fbfa3e000-5f2fbfa3f000 rw-p 00003000 103:06 2756768
/home//IPC/mmap/create_a_map
5f2feeb4f000-5f2feeb70000 rw-p 00000000 00:00 0                          [heap]
7b3a7ac00000-7b3a7ac28000 r--p 00000000 103:05 132776                    /usr/.../libc.so.6
7b3a7ac28000-7b3a7adbd000 r-xp 00028000 103:05 132776                    /usr/.../libc.so.6
7b3a7adbd000-7b3a7ae15000 r--p 001bd000 103:05 132776                    /usr/.../libc.so.6
7b3a7ae15000-7b3a7ae16000 ---p 00215000 103:05 132776                    /usr/.../libc.so.6
7b3a7ae16000-7b3a7ae1a000 r--p 00215000 103:05 132776                    /usr/.../libc.so.6
7b3a7ae1a000-7b3a7ae1c000 rw-p 00219000 103:05 132776                    /usr/.../libc.so.6
7b3a7ae1c000-7b3a7ae29000 rw-p 00000000 00:00 0          /* .bss of shared library */
7b3a7ae68000-7b3a7ae6b000 rw-p 00000000 00:00 0
7b3a7ae7b000-7b3a7ae7d000 rw-p 00000000 00:00 0
7b3a7ae7d000-7b3a7ae7f000 r--p 00000000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7b3a7ae7f000-7b3a7aea9000 r-xp 00002000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7b3a7aea9000-7b3a7aeb4000 r--p 0002c000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7b3a7aeb4000-7b3a7aeb5000 r--p 00000000 103:06 2758042
/home//IPC/mmap/test.txt
7b3a7aeb5000-7b3a7aeb7000 r--p 00037000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7b3a7aeb7000-7b3a7aeb9000 rw-p 00039000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7ffc3905f000-7ffc39081000 rw-p 00000000 00:00 0                          [stack]
7ffc39162000-7ffc39166000 r--p 00000000 00:00 0                          [vvar]
7ffc39166000-7ffc39168000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

When the program terminates, these virtual memory mappings are automatically unmapped.

Or we can use this function to unmap:

```
int munmap(void *addr, size_t length);
/* Returns 0 on success, or –1 on error */
```

Alternatively, we can unmap part of a mapping, is case the mapping either skrinks or is cut in two, depending on where unmapping occurs
During unmapiping, kernel removes any **locks** that the process holds for the specified address range\

7.2 Private/Shared File Mappings

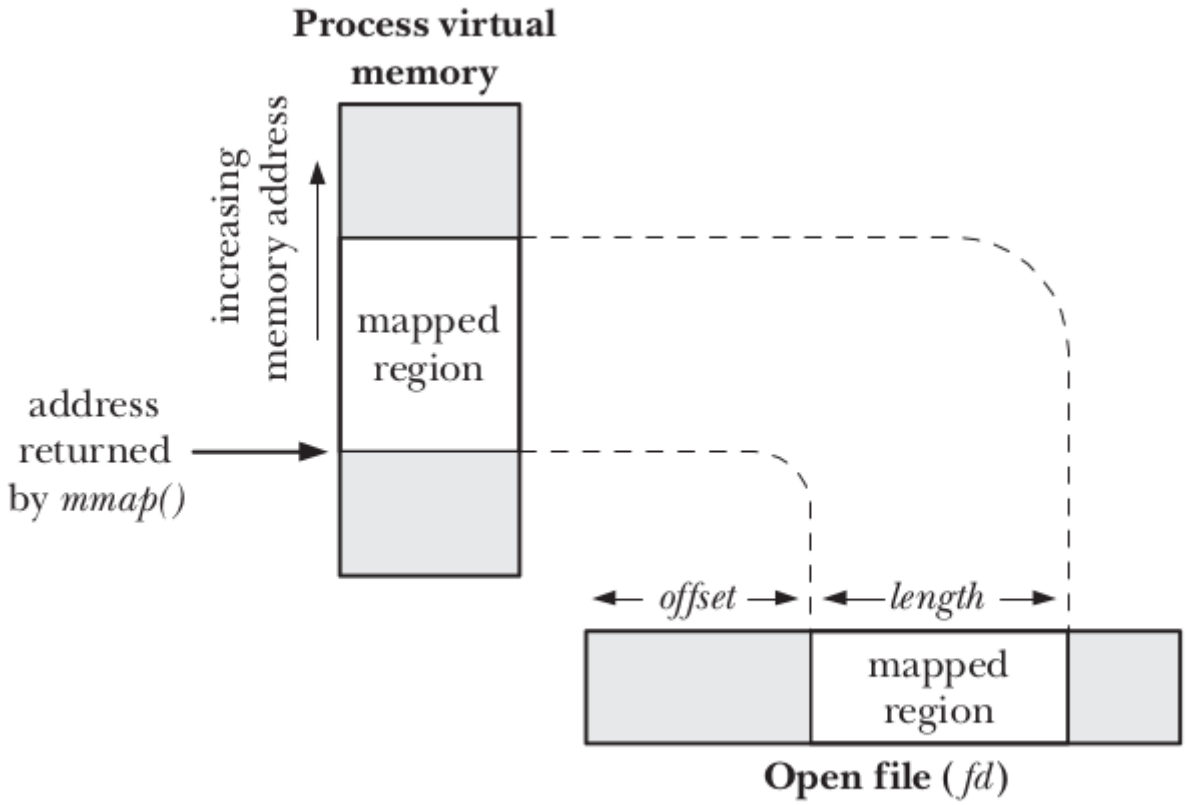


Figure 49-1: Overview of memory-mapped file

NOTE: *offset* mean What position (bytes) we start to read from file?

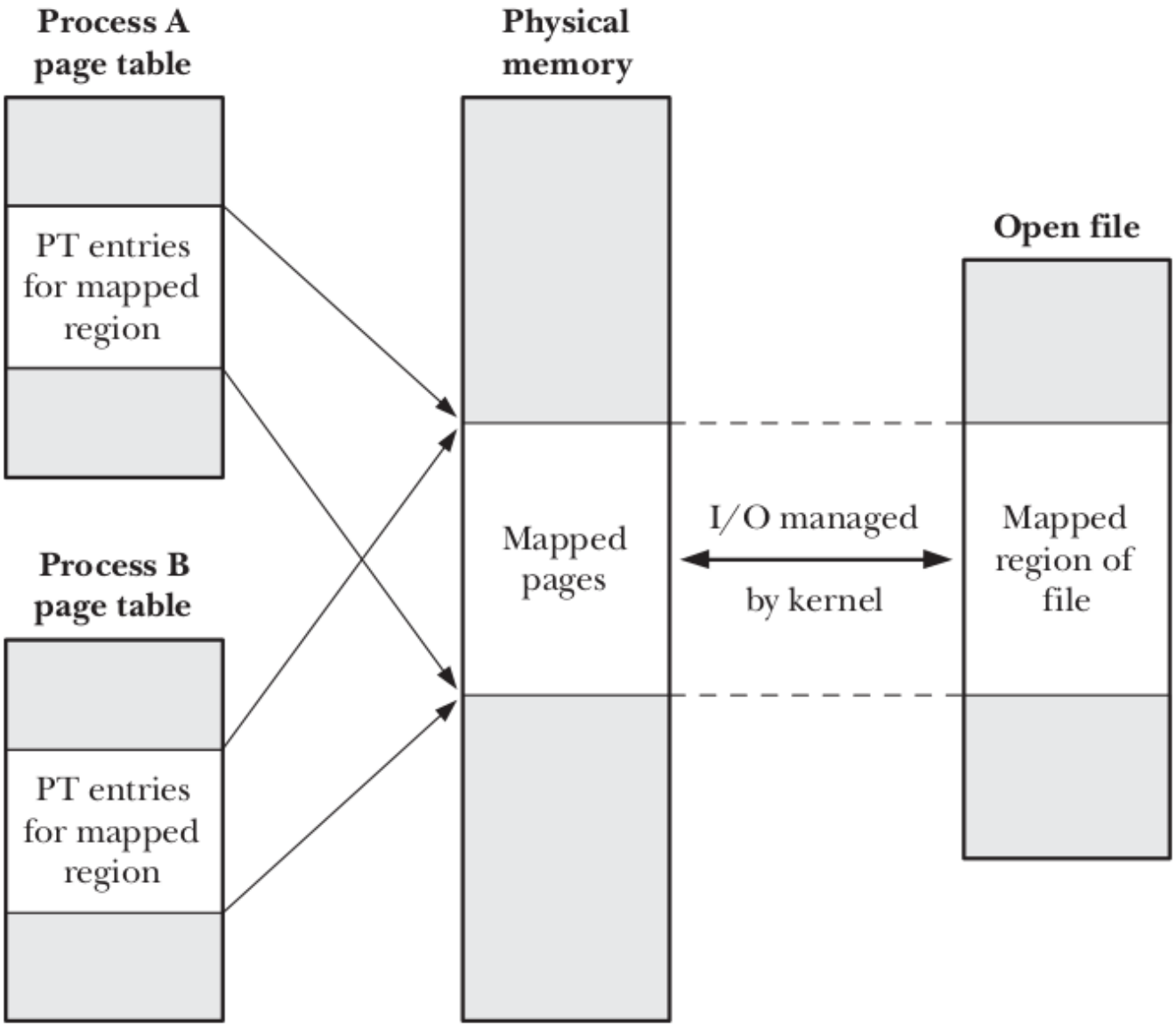


Figure 49-2: Two processes with a shared mapping of the same region of a file

Multiple processes create shared mappings of the same file region, they all share the same physical pages of memory --> modify contents of mapping are carried through to the file

Memory-mapped I/O

Perform file I/O simply by accessing bytes of memory. Kernel ensure that changes to memory are propagated to mapped file

It's same as *read()* and write() to access the contents of a file: (2 advantage)

- replace read() and write() system call, simply logic
- in some circumstances, provide better performance than using I/O system call (Here is **I/O carried out**)

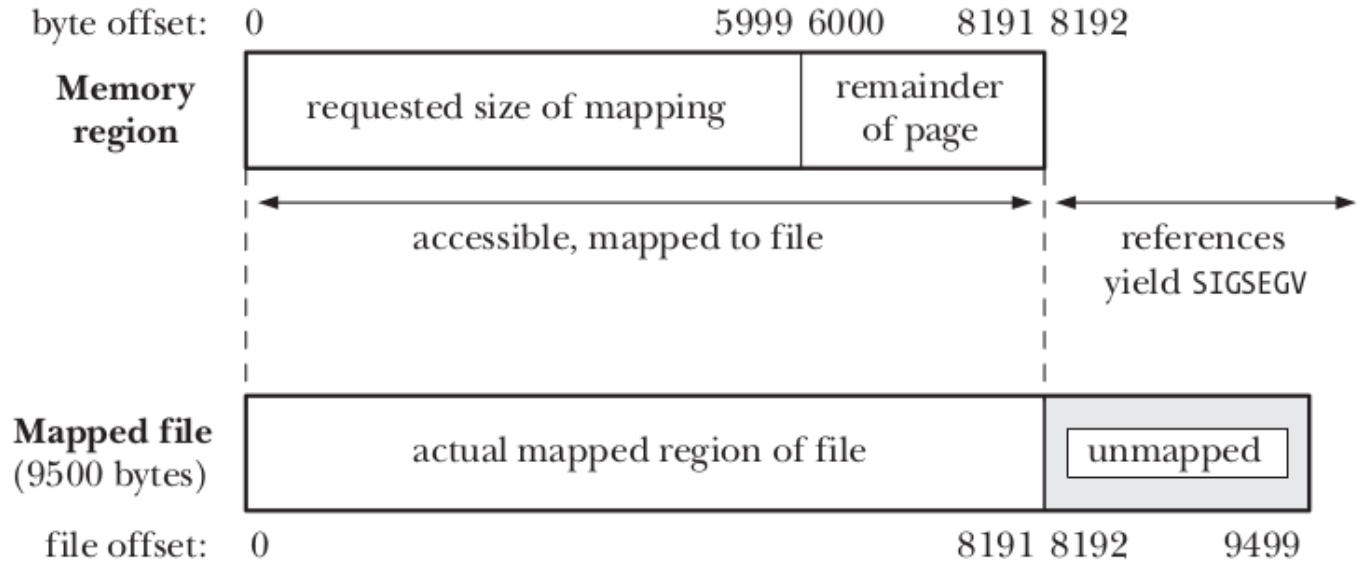
IPC using a shared file mapping

Because we can shared the same physical pages of memory, we can use is as a mothod **fast IPC**. Especially, we can modify the contents of the region are carried through to the underlying mapped file

7.3 Boundary case

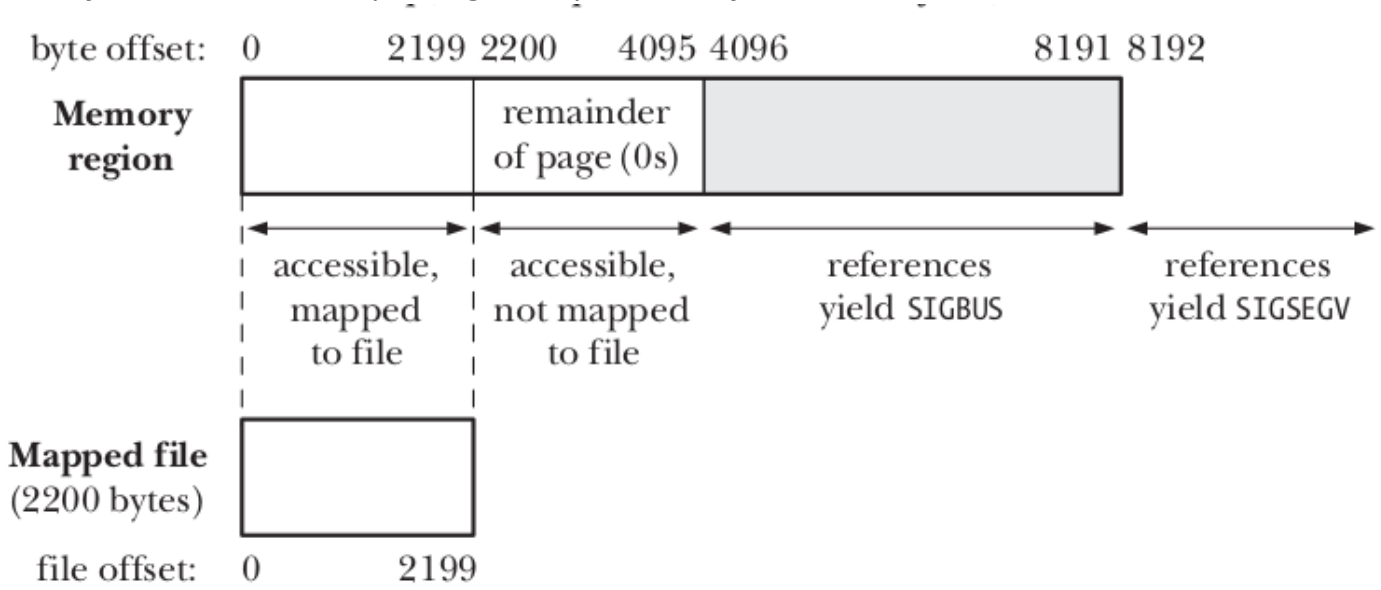
Normally, the size of mapping is a multiple of the system page size, but sometimes not.

Case 1: Size of file is larger than size of mapping (rounded up)
Assump we have file size is: 9500 bytes and the size requesting to map is 6000 byte
Use `mmap(0, 6000, prot, MAP_SHARED, fd, 0);`



If the size of mapping is not multiple of system page size, it's rounded up to the next multiple of system page size
--> It means that we can read **mapped[8191]** although we only require 6000 bytes for size
--> Beside that, if we access beyond the end of the mapping result. It returns SIGSEGV fault

Case 2: Size of mapping extends beyond the end of the underlying file Assump we have file size is: 2200 bytes and the size requesting to map is 8192 bytes



It's rounded up, but In this case:

- bytes in rounded-up region are accessible, they are not mapped to the underlying file. Instead, they are initialized to 0 (**zero-fill**).
- These bytes will sufficiently be shared with other processes mapping the file
- Change to these bytes are not written to the file
  - Example: (my file size is 74 bytes, we require mapping 8192 bytes)

```
int fd = open("test.txt", O_RDONLY);

char *mapped = mmap(NULL, 8192, PROT_READ, MAP_PRIVATE, fd, 0);

if (mapped == MAP_FAILED) {
    printf("Map fail\n");
    return 1;
}

printf("Mapping successfully at virtual address: %p\n", mapped);
printf("73th byte: %c\n", mapped[73]); /* Last byte */
printf("100th byte: %d\n", mapped[100]); /* Out of size file */
printf("4000th byte: %d\n", mapped[4000]); /* Still in rounded up path*/
printf("8200th byte: %d\n", mapped[8200]); /* Out of mapping size*/
printf("8201th byte: %d\n", mapped[8201]);
printf("8202th byte: %d\n", mapped[8202]);
printf("5000th byte:%c", mapped[5000]); /*Out of first rounded up of system page
size*/
```

Result:

```
Mapping successfully at virtual address: 0x7948f67cf000
73th byte: m
100th byte: 0
4000th byte: 0
8200th byte: 109
8201th byte: -99 (random, maybe 13, 77,..)
8202th byte: -16 (random, maybe 40, -19...)
Bus error (core dumped)
```

Why no SIGSEGV fault when access to **OUT OF MAPPING SIZE**

Follow theory of above picture, when print the 8200-8202th byte, it must be return SIGSEGV fault & program is crashed.
But the reality result in the **random result**

For nature, read **mapped[8200] or 8202** is **undefine behavior**, because this address is not have valid **VMA**:

- SIGSEGV - (no page table entry, kernel cant find VMA)
- randomly choose virtual memory is mapped next to --> can be read, no fault Try to print virtual memory range of my program
  Run `cat /proc/20812/maps`
  Result:

```
629ebc3f9000-629ebc3fa000 r--p 00000000 103:06 2752554
/home/.../boundary_case
629ebc3fa000-629ebc3fb000 r-xp 00001000 103:06 2752554
/home/.../boundary_case
629ebc3fb000-629ebc3fc000 r--p 00002000 103:06 2752554
/home/.../boundary_case
629ebc3fc000-629ebc3fd000 r--p 00002000 103:06 2752554
/home/.../boundary_case
629ebc3fd000-629ebc3fe000 rw-p 00003000 103:06 2752554
/home/.../boundary_case
629ef526b000-629ef528c000 rw-p 00000000 00:00 0                          [heap]
7f6228800000-7f6228828000 r--p 00000000 103:05 132776                    /usr/.../libc.so.6
7f6228828000-7f62289bd000 r-xp 00028000 103:05 132776                    /usr/.../libc.so.6
7f62289bd000-7f6228a15000 r--p 001bd000 103:05 132776                    /usr/.../libc.so.6
7f6228a15000-7f6228a16000 ---p 00215000 103:05 132776                    /usr/.../libc.so.6
7f6228a16000-7f6228a1a000 r--p 00215000 103:05 132776                    /usr/.../libc.so.6
7f6228a1a000-7f6228a1c000 rw-p 00219000 103:05 132776                    /usr/.../libc.so.6
7f6228a1c000-7f6228a29000 rw-p 00000000 00:00 0
7f6228abd000-7f6228ac0000 rw-p 00000000 00:00 0
7f6228ace000-7f6228ad0000 r--p 00000000 103:06 2758042                   /home/.../test.txt
7f6228ad0000-7f6228ad2000 rw-p 00000000 00:00 0
7f6228ad2000-7f6228ad4000 r--p 00000000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7f6228ad4000-7f6228afe000 r-xp 00002000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7f6228afe000-7f6228b09000 r--p 0002c000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7f6228b0a000-7f6228b0c000 r--p 00037000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7f6228b0c000-7f6228b0e000 rw-p 00039000 103:05 132761                    /usr/.../ld-linux-
x86-64.so.2
7ffcd2572000-7ffcd2594000 rw-p 00000000 00:00 0                          [stack]
```

```
7ffcd25f4000-7ffcd25f8000 r--p 00000000 00:00 0                          [vvar]
7ffcd25f8000-7ffcd25fa000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

What we have inside **7f6228ad0000-7f6228ad2000** place next to my file **test.txt**
It's anonymous mapping, but what library is belong to?
I use `strace -f -e trace=mmap,brk ./boundary_case` to see What `mmap` belong to it?
Result:

```
brk(NULL)                               = 0x6460ac502000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d37273f9000
mmap(NULL, 63080, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7d37273e9000
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7d3727000000
mmap(0x7d3727028000, 1658880, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x28000) = 0x7d3727028000
mmap(0x7d37271bd000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1bd000) =
0x7d37271bd000
mmap(0x7d3727216000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x215000) = 0x7d3727216000
mmap(0x7d372721c000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0)
= 0x7d372721c000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d37273e6000
mmap(NULL, 8192, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7d37273f7000
brk(NULL)                               = 0x6460ac502000
brk(0x6460ac523000)                     = 0x6460ac523000
Mapping successfully at virtual address: 0x7d37273f7000
73th byte: m
100th byte: 0
4000th byte: 0
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
--- SIGWINCH {si_signo=SIGWINCH, si_code=SI_KERNEL} ---
+++++++++++++++++++++++++++++++++++++++++++++++++++
```

We can see 2 lines: **`mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7d37273f9000`** and
`mmap(NULL, 8192, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7d37273f7000`

NOTE: **brk()** is the extend memory command. It's mean Heap has a end point, call **Program Break**
--> Whenever need to extend memory (higher), use this **system call**

So the reason, `7f6228ad0000-7f6228ad2000 rw-p 00000000 00:00 0` is the result of `mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)`
--> this is **glibc/dynamic linker** call before map my file (We can see it appear sooner than my `mmap`)

I cant confirm certainly who run this system call `mmap()`

Check by:

```
gdb ./boundary_case
(gdb) catch syscall mmap
(gdb) run
(gdb) bt
```

Result:

```
(gdb) run
Starting program: /home/hoainam0508/Documents/IPC/mmap/boundary_case

Catchpoint 1 (call to syscall mmap), __mmap64 (offset=0, fd=-1, flags=34, prot=3, len=8192,
addr=0x0) at ../sysdeps/unix/sysv/linux/mmap64.c:58
58      ../sysdeps/unix/sysv/linux/mmap64.c: No such file or directory.
(gdb) bt
#0  __mmap64 (offset=0, fd=-1, flags=34, prot=3, len=8192, addr=0x0) at
../sysdeps/unix/sysv/linux/mmap64.c:58
#1  __mmap64 (addr=addr@entry=0x0, len=len@entry=8192, prot=prot@entry=3,
flags=flags@entry=34, fd=fd@entry=-1, offset=offset@entry=0) at
../sysdeps/unix/sysv/linux/mmap64.c:46
#2  0x00007ffff7fd0471 in __minimal_malloc (n=320) at ./elf/dl-minimal-malloc.c:59
#3  0x00007ffff7fcb79f in malloc (size=<optimized out>) at ../include/rtld-malloc.h:56
#4  _dl_init_paths (llp=0x0, source=0x0, glibc_hwcaps_prepend=<optimized out>,
glibc_hwcaps_mask=<optimized out>) at ./elf/dl-load.c:739
#5  0x00007ffff7fe610f in call_init_paths (state=0x7fffffffd280) at ./dl-main.h:109
#6  dl_main (phdr=<optimized out>, phnum=<optimized out>, user_entry=<optimized out>,
auxv=<optimized out>) at ./elf/rtld.c:1728
#7  0x00007ffff7fe283c in _dl_sysdep_start (start_argptr=start_argptr@entry=0x7fffffffd5f0,
dl_main=dl_main@entry=0x7ffff7fe48e0 <dl_main>) at ../elf/dl-sysdep.c:256
#8  0x00007ffff7fe4598 in _dl_start_final (arg=0x7fffffffd5f0) at ./elf/rtld.c:507
#9  _dl_start (arg=0x7fffffffd5f0) at ./elf/rtld.c:596
#10 0x00007ffff7fe3298 in _start () from /lib64/ld-linux-x86-64.so.2
```

We can see line 2, it's the main function run `mmap` above
--> Not .bss or .data segment. It's **scratch local memory** which dynamic linker (**ld.so**) self-allocate in **bootstrap** phase/stage, before **`glibc`** is loaded into program

msync(): Synchonizing a mapped region\

Problem: modify contents of the MAP_SHARED mapping through to the underlying file, but not guarantees about synchonization
solution: give an application explicit control over when a shared mapping is synchonized with the mapped file

Prototype:

```
#include <sys/mman.h>
int msync(void *addr, size_t length, int flags);
                          /* Returns 0 on success, or –1 on error */
```

ANONYMOUS MAPPING:
An anonymous mapping is one that doesn’t have a corresponding file
Use **MAP_ANONYMOUS** flag in mmap() command to create an anonymous mapping\

- The bytes of the result mapping are initialized to 0

A **MAP_SHARED anonymous mapping** allows related processes (e.g., parent and child) to share a region of memory without needing a corresponding mapped file.

Flags	VMA (Merge)?	Reality Max
MAP_PRIVATE / MAP_ANONYMOUS	Yes (absolutely)	Vitual memory size (128TB).
MAP_SHARED / MAP_ANONYMOUS	No	Number Max (~65,530).
File-backed (MAP_SHARED/PRIVATE)	No	Number Max (~65,530).

We can change the size of mapping region through:

```
#define _GNU_SOURCE
#include <sys/mman.h>
void *mremap(void *old_address, size_t old_size, size_t new_size, int flags, ...);
                      /* Returns starting address of remapped region on success, or MAP_FAILED
on error */
```

8. Semaphore (POSIX)

8.1 Named Semaphores

Openning a Named Semaphore:
Function creates and opens a new named semaphore or opens an exitsting semaphore
Prototype:

```
#include <fcntl.h>       /* Defines O_* constants */
#include <sys/stat.h>    /* Defines mode constants */
#include <semaphore.h>
sem_t *sem_open(const char *name, int oflag, ...
                                          /* mode_t mode, unsigned int value */ );
                                      /* Returns pointer to semaphore on success, or
SEM_FAILED on error */
```

- *name*: identifies the semaphore
- *oflag*:
  - 0: accesing an existing semaphore
  - O_CREAT: new semaphore is created if *name* doesn't exist
  - O_CREAT | O_EXCL: create exclusive semaphore If *oflag* differ from 0, we have more 2 flags:
- *mode*: bit mask (O_RDONLY, O_WRONLY, and O_RDWR)
- value: specifies the initial value to be assignd to the new semaphore

Closing a Semaphore: terminate the association betwwen the process and the semaphore

```
#include <semaphore.h>
int sem_close(sem_t *sem);
                                                          /* Returns 0 on success, or –1 on
error */
```

Removing a Named Semaphore: remove the semaphore identified by *name* and marks the semaphore to be destroyed once all processes cease

Example:

```
sem_t *sem = sem_open("/my_sem", O_CREAT , 0644, 1);
    if (sem == SEM_FAILED) {
        printf("Error create Sem\n");
        exit(1);
    }
    else printf("Create semaphore successfully!\n");
```

```
    int val;
    sem_getvalue(sem, &val);
    printf("Semaphore value = %d\n", val);

    if (sem_close(sem) == -1) {
        printf("Error close Sem\n");
        exit(1);
    }
```

Result:

```
Create semaphore successfully!
In critical section
Semaphore value = 1
```

On linux, we can check semaphore exist through `ls -l /dev/shm/sem.[name]`.
Result

```
-rw-r--r-- 1 hoainam0508 hoainam0508 32 Thg 9  21 16:44 sem.my_sem
-rw-r--r-- 1 hoainam0508 hoainam0508 32 Thg 9  21 16:52 sem.my_sem2
-rw-r--r-- 1 hoainam0508 hoainam0508 32 Thg 9  21 16:53 sem.my_sem3
```

Waiting on a Semaphore: decrements (decreases by 1) the value of the semaphore referred to by *sem*
Prototype:

```
#include <semaphore.h>
int sem_wait(sem_t *sem);
```

if semaphore currently has a value greater than 0, sem_wait() return immediately
If the value is currently 0, block until semaphore value rises above 0.

The *sem_trywait()* function is a nonblocking version of *sem_wait().*

```
#include <semaphore.h>
int sem_trywait(sem_t *sem);
```

If the decrement operation can’t be performed immediately, *sem_trywait()* fails with the error EAGAIN\

The *sem_timedwait()* function is another variation on *sem_wait()*. It allows the caller to specify a limit on the time for which the call will block.

```
#define _XOPEN_SOURCE 600
#include <semaphore.h>
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

Posting a Semaphore
The *sem_post()* function increments (increases by 1) the value of the semaphore referred to by *sem*

```
#include <semaphore.h>
int sem_post(sem_t *sem);
```

8.2 Unnamed Semaphore

Overview:

- type *sem_t*, are stored in memory allocated by the application
- placing this semphore in an area of memory which processes and threads share

Unnamed vs Named semaphores:

- sem shared between processes doesn't need a name. Making an unnamed semaphores a shared (global or heap) variale automatically makes it accessible to all threads
- same with processes, If parent allocates an unnamed semaphore in a region of shared memory, then child automatically inherits the mapping and thus the semaphore as part of the operation of fork()

initializing:

```
#include <semaphore.h>
int sem_init(sem_t *sem, int pshared, unsigned int value);
                                                        /* Returns 0 on success, or –1
on error */
```

- pshared: semaphore shared between thread or processes?
  - = 0: between threads of the calling process, *sem* is specified as the address of global variable or variable allocated on the heap. Terminate whenever process terminate
  - non-zero: shared between processes, *sem* must be the address of a location in a region of shared memory. Persist as long as the shared memory in which it resides **destroy**: It is safe to destroy a semaphore only if no processes or threads are waiting on it

```
#include <semaphore.h>
int sem_destroy(sem_t *sem);
                                    /* Returns 0 on success, or –1 on error */
```

Example:

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t sem_empty;
sem_t sem_full;
int   buffer;      // local variable

void *producer(void *arg)
{
    for (int i = 1; i <= 5; i++) {
        sem_wait(&sem_empty);   // wait for empty
        buffer = i;
        printf("[Producer] produced %d\n", i);
        sem_post(&sem_full);    // inform item ready
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 5; i++) {
        sem_wait(&sem_full);    // wait for item
        printf("[Consumer] consumed %d\n", buffer);
        sem_post(&sem_empty);   // inform slot is empty
        usleep(150000);
    }
    return NULL;
}

int main(void)
{
```

```
    sem_init(&sem_empty, 0, 1);  // thread shared, has 1 slot
    sem_init(&sem_full, 0, 0);   // no has any item

    pthread_t t_prod, t_cons;
    pthread_create(&t_prod, NULL, producer, NULL);
    pthread_create(&t_cons, NULL, consumer, NULL);

    pthread_join(t_prod, NULL);
    pthread_join(t_cons, NULL);

    return 0;
}
```

Result:

```
[Producer] produced 1
[Consumer] consumed 1
[Producer] produced 2
[Consumer] consumed 2
[Producer] produced 3
[Consumer] consumed 3
[Producer] produced 4
[Consumer] consumed 4
[Producer] produced 5
[Consumer] consumed 5
```

Semaphore Litmits:

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ getconf SEM_VALUE_MAX
2147483647      /* max of counter */

hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ ulimit -n
1024     /* max of sem_open() */
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ getconf NAME_MAX /dev/shm
255      /* max of char in name */
```

9. Shared memory POSIX

Overview: allows to us to share a mapped region between unrelated processes without needing to create a corresponding mapped file Two steps for use a POSIX shared memory:

- use *shm_open()* function to create or open an exist object with a specified name --> return a file descriptor referring to the object
- pass this fd in to *mmap()* that specified **MAP_SHARED** in *flag* argument --> this maps the shared memory object into the process's virtual address space

Creating Shared Memory Objects

```
#include <fcntl.h>                      /* Defines O_* constants */
#include <sys/stat.h>                   /* Defines mode constants */
#include <sys/mman.h>
int shm_open(const char *name, int oflag, mode_t mode);
                                            /* Returns file descriptor on success, or –1 on
error */
```

flag:

Table 54-1: Bit values for the *shm_open() oflag* argument

Flag	Description
O_CREAT	Create object if it doesn't already exist
O_EXCL	With O_CREAT, create object exclusively
O_RDONLY	Open for read-only access
O_RDWR	Open for read-write access
O_TRUNC	Truncate object to zero length

Example:

```
#define SHM_SIZE 4096     //  page size

int main(void)
{
    int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0644);
    if (fd == -1) {
        printf("Erorr create shared memory object\n");
        exit(1);
    }

    /* Allocate reality size*/
```

```
    if (ftruncate(fd, SHM_SIZE) == -1) {
        printf("Erorr truncate shared memory object\n");
        exit(1);
    }

    void *ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) {
        printf("Erorr map shared memory object\n");
        exit(1);
    }

    close(fd);

    pid_t pid = fork();
    if (pid == 0) {
        sleep(1);
        printf("[Child] read : %s\n", (char *)ptr);
        exit(1);
    } else {
        strcpy((char *)ptr, "Hello from parent via shared memory!");
        printf("[Parent] Wrote into shared memory\n");
        wait(NULL);
    }
    return 0;
}
```

Compared between Share Memory

10. Signals

10.1 Overview

Overview:

- A signal is a notification to a process that an event has occurred
- sometimes described as *software interrupts*

- Signals are analogous to hardware interrupts in that they interrupt the normal flow of execution of a program
- One process can send a signal to another process.
- It is also possible for a process to send a signal to itself
- **Ctrl-C** (interrupt) & **Ctrl-Z** (suspend)
- Signals has 2 categories: *traditional* and *standard*
- On Linux, standard signals are numbered from 1 to 31
- Signal is *generate* by some event. Once generated, a signal is later *delivered* to a process. Between the time it is generated and the time it is delivered, a signal is said to be *pending*
- *signal mask* - a set of signals whose delivery is currently *blocked* --> ensure that a segment of code is not interrupted by the delivery of a signal

Default action of process (related Signals):

- The signal is *ignored*, it is discarded by the kernel and has no effect on the process
- The process is *terminated* (killed). We call *abnormal process termination*, **as opposed to** it that occur when a process terminate call `exit()`
  - exit(): inside, process self-call && `clean function + flush stdio buffer + grateful end`
  - killed: outside, no option to change && `no clean function + no flush buffer + Abrupt end`
- *core dump file* signal: contains an image of the virtual memory of the process
- *stopped* (kill -20 | ctrl Z): process is suspend
- Execution of the process is resumed after previously being stopped.

Instead of accepting the default for a particular signal, a program can change the action that occurs when the signal is delivered. This is known as setting the **disposition of the signal** Can set:

- *default action* should occur. This is useful to undo an earlier change of the disposition of the signal to something other than its default.
- The signal is *ignored*. This is useful for a signal whose default action would be to terminate the process.
- A *signal handler* is executed.

Note that it isn't possible to set the disposition of a signal to *terminate* or *dump core*

10.2 Usage

We can **changing signal dispositions**:

```
#include <signal.h>
void ( *signal(int sig, void (*handler)(int)) ) (int);
                                /* Returns previous signal disposition on success, or SIG_ERR
```

```
on error */
```

- *sig*: identifies the signal whose disposition we wish to change
- *handler*: is the address of the function that should be called when this signal is delivered

Introduction to Signal Handlers: Definition: A *signal handler* (also called a *signal catcher*) is a function that is called when a specified signal is delivered to a process

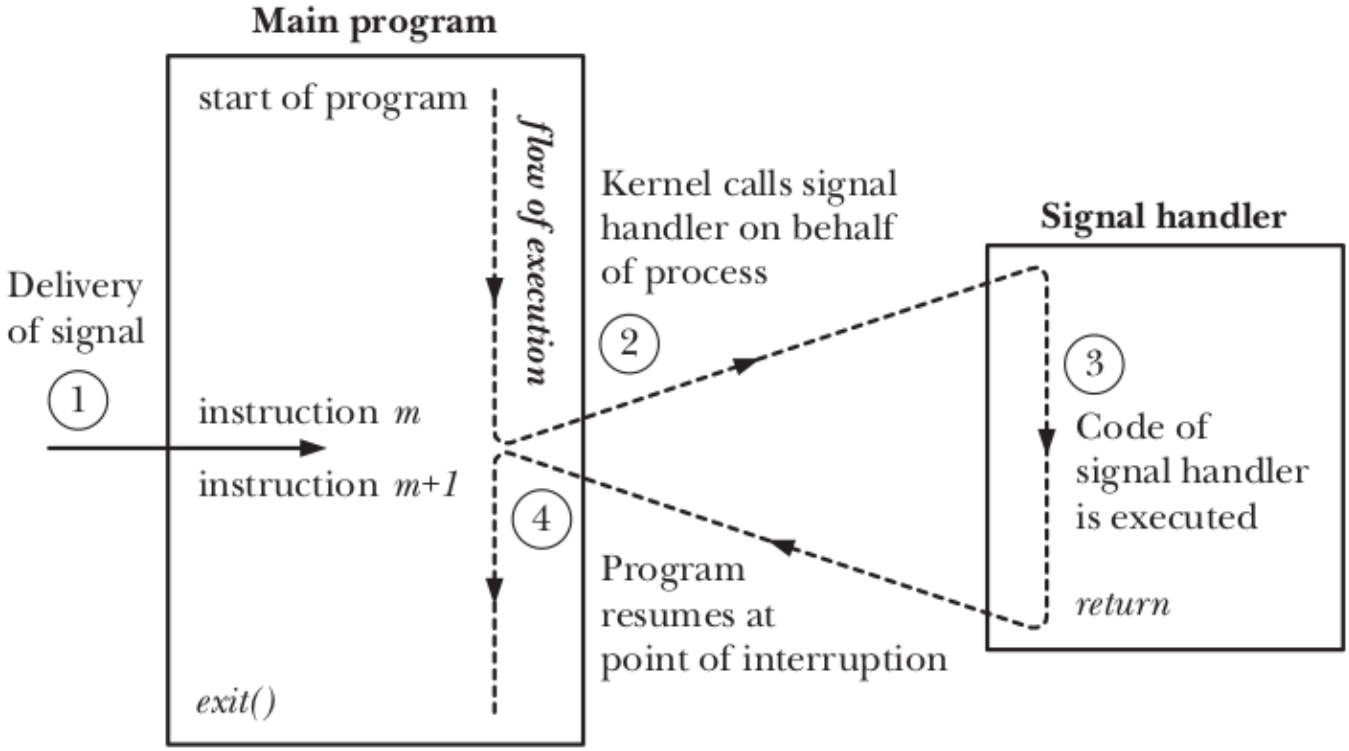


Figure 20-1: Signal delivery and handler execution

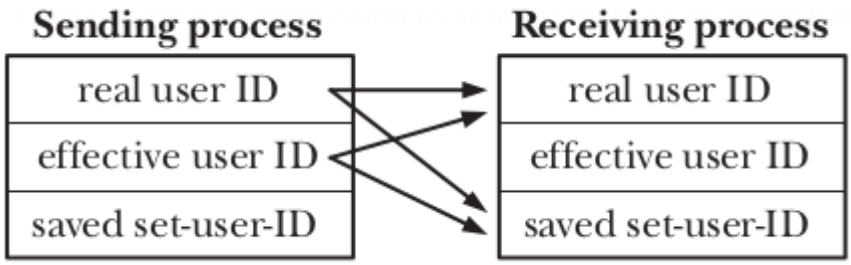
Sending Signals: One process can send a signal to another process

```
#include <signal.h>
int kill(pid_t pid, int sig);
/* Returns 0 on success, or –1 on error */
```

- pid: identifies one or more processes to which the signal specified by *sig* is to be sent (4case)
  - greater than 0: signel is sent to process has Process ID = pid
  - = 0: the signal is sent to every process in the same process group as the calling process, including the calling process itself
  - < -1: sent to all of the processes in the process group whose ID = abs(pid)
  - = -1: send to all process, except init (PID = 1) & calling process

A process needs appropriate permissions to be able send a signal to another process:

- A privileged (CAP_KILL) process may send a signal to any process.

- Process ID 1, which runs with user and group of root, is a special case. It can be sent only signal which it has a **handler installed**. This prevents the **system administrator** from accidentally killing *init* (**PROTECT MECHANISM**)
- if IDs mat 

ivileged process)

- SIGCONT: An unprivileged process may send this signal to any other process in the same session

> **NOTE: If process want to send signal to another process, it must have the permission**

Checking for the Existence of a Process
Will kill() system call, If *sig* argument is specified as 0, then no signal is sent.
Example: `int kill(pid_t pid, 0);`
This mean we can use the null signal to test if a process with specific process ID exists

- Return ESRCH: process doesn't exist
- Return EPERM: process exits. but dont have permission to send a signal to it
- succeed: have permission to send signal (or process exists)

We can use `int raise(int sig)` to send a signal to itself (for process or thread)
But with thread, raise() contrast with kill():

- raise(): signal will be delivered to the specific thread that called raise()
- kill(getpid(), sig) sends a signal to caling *process*, that signal may be delivered to **any thread** in the process.

We can send a signal to all of the member of a *process group*

```
#include <signal.h>
int killpg(pid_t pgrp, int sig);
```

This same with `kill(-pgrp, sig);`
If pgrp = 0, send to all process of process group as the caller

Displaying Signal Descriptions:
Each signal has an associated printable description. Use strsignal() function

```
#define _GNU_SOURCE
#include <string.h>


char *strsignal(int sig);
                                                /* Returns pointer to signal description
string */
```

Signal set: Many signal-related system calls need to be able to represent a group of different signals

- avoid specified signal: `int sigemptyset(sigset_t *set);`
- permit specified signal: `int sigfillset(sigset_t *set);`
  After initialization, individual signals can be added to a set using sigaddset() and removed using sigdelset().

```
#include <signal.h>
int sigaddset(sigset_t *set, int sig);
int sigdelset(sigset_t *set, int sig);
```

The sigismember() function is used to test for membership of a set.

```
#include <signal.h>
int sigismember(const sigset_t *set, int sig);
```

The Signal Mask (Blocking Signal Delivery) If a signal that is blocked is sent to a process, delivery of that signal is delayed until it is unblocked by being removed from the process signal mask. To a signal can be added to signal mask:

- When a signal handler is invoked, automatically added to the signal mask
- When a signal handler is established with sigaction(), it is possible to specify an additional set of signals that are to be blocked when the handler is invoked

```
#include <signal.h>
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

We can use sigprocmask() to change the process signal mask, to retrieve the existing mask, or both.

Pending Signal: To determine which signals are pending for a pro- cess, we can call sigpending().

```
#include <signal.h>
int sigpending(sigset_t *set);
```

If we change the disposition of a pending signal, then, when the signal is later unblocked, it is handled according to its new disposition

Signals Are Not Queued (Standard Signals)

Changing Signal Dispositions

- more complex than signal() because we can retrive the disposition of a signal without changing it
- more portable than signal() thwn establishing a signal handler

```
#include <signal.h>
int sigaction(int sig, const struct sigaction *act, struct sigaction *oldact);
                                                    /* Returns 0 on success, or –1 on error
*/
```

- sig: identifies the signal whose disposition we want to retrive or change (any signal except SIGKILL and SIGSTOP)
- act: pointer to a structure specifying a new disposition for the signal (use NULL if existing disposition of the signal)
- oldact: return information about the signal's previous disposition

Waiting for a Signal

```
#include <unistd.h>
int pause(void);
                                                /* Always returns –1 with errno set to EINTR */
```

pros: no cost CPU performance

10.3 Signal Handlers

10.3.1 **Designing Signal Handlers**

Two common design for signal handlers:

- Set a glocal flag and exits. The main program periodically check this flag and, if it is set, takes appropriate action
- performs some type of cleanup and then either terminates the process or uses a nonlocal goto to unwind the stack and return control to a predetermined location in the main program

Signal are not queued: If signal is generated while the handler is executing, then it is marked as pending and later delivered when the handler return (still pending if generated more than one signal)

Reentrancy and Async-Signal-Safe functions

- Reentrant function: it can safely be simultaneously executed by multiple threads of execution in the same process
- nonreentrant: if 2 invocation of the function simultaneously attempt to update the same global variable or data structure --> these update are likely to interfere with each other and produce incorrect results

Standard async-signal-safe functions: is one that implements guarantees to be safe when called from a signal handler. A function is async-signal-safe because:

- it's reentrant
- or it's not interuptible by a signal handler

10.3.2 Terminatating a Signal Handler

some methods

- use _exit() to terminate the process.
- use kill() or *raise()* to send a signal that kills the process
- Perform a nonlocal *goto* from the signal handler
- Use the abort() function to terminate the process with a core dump

Terminating a Process Abnormally
abort() function terminates the calling process and causes it to produce a core dump

```
#include <stdlib.h>
void abort(void);
```

It terminates calling process by raising a SIGABRT signal

10.3.3 Handling a Signal on an Alternate Stack

Normally, when a signal handler is invoked, the kernel creates a frame for it on the process stack
However, this may not be possible if a process attempts to extend the stack beyond the maximum possible size
--> It may occur stack grows so large that it encounters a region of mapped memory or the

upwardly growing heap, or it reaches the RLIMIT_STACK resource limit

When grow to maximum possible size, kernel generates a SIGSEGV signal for process

But If stack space is exhausted, the kernel can’t create a frame for any SIGSEGV handler that the program may have established

If we instead need to ensure that the SIGSEGV signal is handled in these circumstances, we can do the following:

- Allocate an area of memory, called an alternate signal stack, to be used for the stack frame of a signal handler.
- Use the sigaltstack() system call to inform the kernel of the existence of the alternate signal stack.
- When establishing the signal handler, specify the SA_ONSTACK flag, to tell the kernel that the frame for this handler should be created on the alternate stack.

```
#include <signal.h>
int sigaltstack(const stack_t *sigstack, stack_t *old_sigstack);
```

- sigstack: points to a structure specifying the location and attributes of the new alternate signal stack
- old_sigstack: points to a structure used to return information about the previously established alternate signal stack

```
typedef struct {
void *ss_sp;            /* Starting address of alternate stack */
int ss_flags;           /* Flags: SS_ONSTACK, SS_DISABLE */
size_t ss_size;         /* Size of alternate stack */
} stack_t;
```

- SS_ONSTACK: indicates that the process is currently executing on the alternate signal stack
- SS_DISABLE: ndicates that there is no currently established alternate signal stack

10.4 Realtime Signal

Overview:
Only two standard signals are freely available for application-defined purposes: SIGUSR1, SIGUSR2
Realtime signals are queued
If multiple instances of a realtime signal are sent to a process, then the signal is delivered multiple times (Standard signal handle only once)
When sending a realtime signal, it is possible to specify data that accompanies the signal. The

signal handler in the receiving process can retrieve this data
Signals are prioritized, with lower-numbered signals having higher priority

Limits on the number of queued realtime signals

```
hoainam0508@hoainam0508-ThinkPad-T14s-Gen-4:~$ ulimit -i
61781           /* max number of queued realtime signals */
```

Sending Realtime Signals:

```
#define _POSIX_C_SOURCE 199309
#include <signal.h>
int sigqueue(pid_t pid, int sig, const union sigval value);
```

Revision #19
Created 2026-09-11 01:42:39 UTC by Lê Hoài Nam
Updated 2026-09-23 01:34:01 UTC by Lê Hoài Nam